IEEE Access
Multidisciplinary | Rapid Review | Open Access Journal

Received 5 February 2026, accepted 27 February 2026, date of publication 2 March 2026, date of current version 6 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3669772

RESEARCH ARTICLE

Multi-Objective Cloud Workflow Scheduling With Two-Stage Deep Reinforcement Learning and Graph Neural Networks

SUNERA CHANDRASIRI AND DULANI MEEDENIYA, (Senior Member, IEEE)
Department of Computer Science and Engineering, University of Moratuwa, Moratuwa 10400, Sri Lanka
Corresponding author: Dulani Meedeniya (dulanim@cse.mrt.ac.lk)

ABSTRACT Cloud workflow scheduling is a critical challenge in modern cloud computing, requiring intelligent task-to-resource assignment under dynamic conditions and conflicting objectives such as makespan, energy consumption, and QoS latency. Traditional methods often fall short in addressing complex task dependencies and multi-objective trade-offs. This paper proposes 2SD-GAT, a novel dynamic scheduling approach that combines Two-Stage Deep Reinforcement Learning (DRL) with Graph Neural Networks (GNNs) to optimize cloud workflow execution. The method models workflows as Directed Acyclic Graphs (DAGs) and uses a Proximal Policy Optimization (PPO)-based two-stage agent to sequentially select tasks and allocate virtual machines (VMs). A Graph Attention Network (GAT) encoder captures workflow structure, while a preference-based reward function enables trade-off exploration between makespan, energy consumption, and QoS latency. The agent is trained on synthetic and real-world datasets. Comparative evaluation against random, heuristic, and evolutionary baselines show that the proposed approach consistently outperforms in hypervolume and IGD metrics while maintaining low decision latency. Notably, it achieves a 26.8% hypervolume gain with a 4.5x lower IGD on real-world cloud traces under dynamic conditions, demonstrating both effectiveness and scalability in real-time multi-objective cloud scheduling.

INDEX TERMS Cloud computing, cloud workflow scheduling, deep reinforcement learning, graph neural networks, multi-objective optimization, Pareto analysis.

I. INTRODUCTION

Cloud computing has become a transformative paradigm, offering scalable and flexible computing resources that drive technological progress across diverse industries [2]. However, the rapid expansion of cloud services has intensified energy demands, with data centers consuming significant electricity globally [3]. Efficient resource management is therefore critical not only for performance but also for sustainability.

Virtualization plays a central role in this landscape, enabling the execution of diverse workloads on shared infrastructure. Among these, workflow applications—modeled as Directed Acyclic Graphs (DAGs)—present unique scheduling challenges due to their complex inter-task dependencies [4]. To manage these workloads, cloud platforms rely on schedulers to assign tasks to Virtual Machines (VMs) while optimizing for multiple, often conflicting objectives such as makespan, energy consumption, and latency [1].

Scheduling in cloud systems is an NP-hard combinatorial optimization problem [5]. Traditional approaches, such as static heuristics and meta-heuristics, often struggle to adapt to the dynamic nature of modern cloud environments. These methods typically assume complete prior knowledge of tasks and resources, which is rarely available in practice due to runtime uncertainties like variable execution times and resource failures [6], [7]. Furthermore, existing solutions fail to adequately address the complex dependencies inherent

The associate editor coordinating the review of this manuscript and approving it for publication was Mehdi Sookhak.

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License.
For more information, see https://creativecommons.org/licenses/by/4.0/

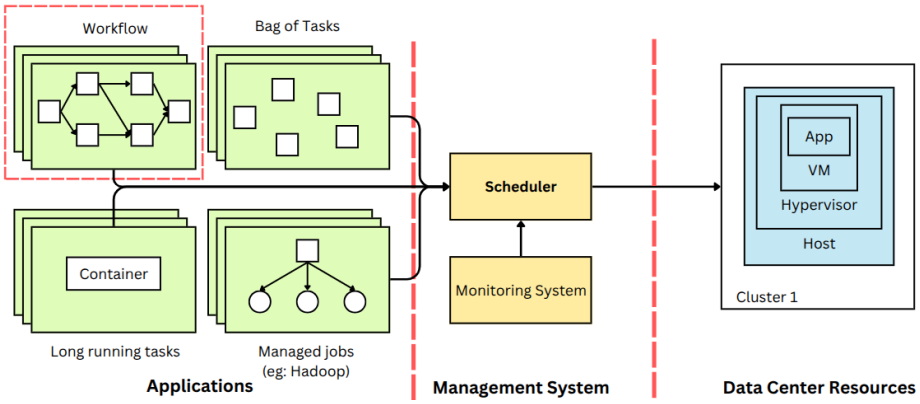


FIGURE 1. System model for data center operation. Figure reproduced from [1].

in DAG-based workflows while simultaneously balancing multi-objective goals. This limitation highlights the need for adaptive, intelligent scheduling strategies capable of navigating the trade-offs between performance and energy efficiency in real-time.

Cloud scheduling also involves multiple, often conflicting objectives: minimizing makespan, energy consumption, latency, cost, and more [8]. Among these, makespan and energy consumption are particularly critical. As cloud infrastructure grows and energy-intensive AI workloads rise [9], optimizing power usage becomes a key concern in line with global sustainability goals such as the Paris Agreement [10], [11].

Recent research explores Deep Reinforcement Learning (DRL) as a promising solution to dynamic scheduling problems [12]. In DRL, agents learn optimal task placement strategies by interacting with the environment and receiving feedback. While existing DRL applications often target independent tasks, workflows demand multi-level decision-making, including workflow selection, task sequencing, and VM assignment. Recently, GNNs have gained attention for their capability to capture dependencies in graph-structured data. While the studies integrating GNNs with DRL for workflow scheduling in cloud environments remain limited, GNNs have demonstrated success in related fields such as the Flexible Job Shop Scheduling Problem (FJSSP), where they effectively model complex task dependencies [13], [14], [15]. This suggests that combining GNNs with DRL could enhance decision-making in dynamic workflow scheduling, enabling optimized schedules that respect task dependencies and execution constraints.

Cloud workflow scheduling often involves Multi-Objective Optimization (MOO), balancing trade-offs between performance, energy efficiency, and other objectives. Instead of a single optimal solution, MOO seeks a Pareto front of non-dominated solutions [16]. Metrics such as Hypervolume (HV) and Inverted Generational Distance (IGD) evaluate the quality and diversity of solutions [17].

Our main novel contribution is the combination of a DRL agent with a GNN for workflow scheduling. This unique integration leverages the structural learning capabilities of GNNs to encode complex task dependencies within the DRL framework. To effectively handle the scheduling complexity, we employ a two-stage decision-making approach, decomposing the problem into task selection and VM assignment. This two-stage formulation is particularly beneficial as it addresses the large action space problem inherent in simultaneous task-resource mapping, which often hinders convergence in flat DRL models. By decoupling the decision process, the agent can learn more efficient policies with better convergence properties, mitigate the curse of dimensionality, and enable effective sequential decision-making in dynamic cloud environments. Specifically, the proposed approach integrates a GNN encoder with a two-stage PPO-based DRL agent, leveraging preference-based rewards to navigate trade-offs among makespan, energy consumption, and QoS latency in real time.

This study is guided by the following research questions (RQs):

- RQ1: How does the proposed two-stage DRL framework, by decomposing task selection and VM assignment, mitigate the dimensionality curse and improve convergence compared to traditional heuristic and meta-heuristic baselines?
- RQ2: Can a preference-based multi-objective reward formulation effectively navigate the trade-offs between conflicting objectives—makespan, energy consumption, and QoS latency—to generate a diverse set of Pareto-optimal solutions?
- RQ3: To what extent does the integration of a GNN encoder enable the effective representation of complex workflow dependencies and enhance the agent's generalization capability across diverse DAG structures?

II. LITERATURE REVIEW

Cloud workflow scheduling has been addressed using a wide range of algorithmic paradigms, from classical heuristics and metaheuristics to recent learning-based methods. These studies differ in how they model workflow dependencies, deal with runtime uncertainty, and handle multi-objective

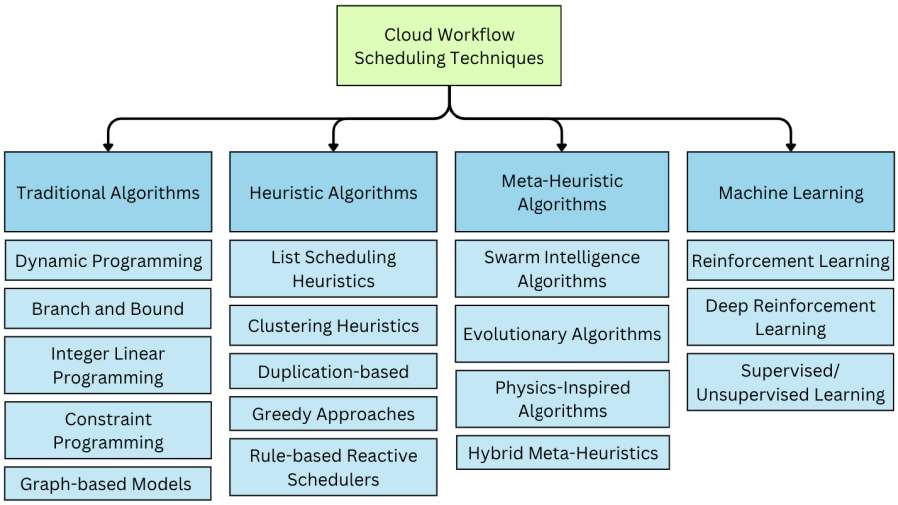


FIGURE 2. Overview of cloud workflow scheduling algorithm taxonomy.

trade-offs such as makespan, cost, energy consumption, and quality of service (QoS). In this section, we focus on representative studies that are directly relevant to dynamic, multi-objective workflow scheduling, and position our two-stage DRL + GNN approach with respect to them.

A. CLASSICAL AND HEURISTIC WORKFLOW SCHEDULING

Early work on workflow scheduling in heterogeneous systems is dominated by deterministic heuristics that assume full knowledge of the workflow and resources. A key reference is the Heterogeneous Earliest Finish Time (HEFT) algorithm [18], which ranks DAG tasks using an upward rank metric and then assigns each task to the VM that minimizes its earliest finish time. HEFT is widely used as a baseline due to its simplicity and strong makespan performance on heterogeneous platforms, but it overlooks energy consumption and other optimization objectives.

Several variants extend HEFT-style list scheduling to incorporate additional constraints. IC-PCP [19] uses partial critical paths to allocate tasks under a global workflow deadline, aiming to minimize monetary cost on pay-as-you-go cloud resources. MOHEFT [20] and FDHEFT [21] further extend list scheduling into a multi-objective setting, jointly optimizing makespan and cost using weighted trade-offs or fuzzy dominance strategies during task ranking and resource selection. However, despite these extensions, HEFT-based approaches remain fundamentally limited by their static priority rules, making them less effective in highly dynamic environments or when navigating complex multi-objective trade-offs.

Energy- and sustainability-aware heuristics have also been explored. MHRA [22] combines several classical scheduling rules to jointly minimize makespan and energy consumption. CETSA [23] tackles cost, energy, and makespan simultaneously in dynamic environments, while RMREC [24] focuses on remapping critical tasks under Dynamic Voltage and Frequency Scaling (DVFS) to reduce energy usage under a budget constraint. General-purpose cluster schedulers such as Tetris [25] and FERPTS [26] provide strong baselines on multi-resource packing and energy efficiency, but they are not natively DAG-aware and typically target job-level rather than workflow-level optimization.

Overall, these heuristic studies provide efficient and often competitive schedules for specific objective sets. However, they typically produce a single solution per run, are difficult to tune for many conflicting objectives, and struggle to adapt to dynamic conditions such as workload bursts, VM failures, or runtime uncertainty.

B. METAHEURISTIC AND EVOLUTIONARY MULTI-OBJECTIVE SCHEDULERS

To better cope with the combinatorial nature of workflow scheduling and the presence of conflicting objectives, many studies adopt metaheuristic search. Particle Swarm Optimization (PSO) has been applied extensively: Pandey et al. [27] minimize execution and data transfer costs, while Rodriguez and Buyya [28] target cost minimization under deadline constraints for scientific workflows. Later PSO variants such as IMPSO [29] and HPSO [30] incorporate domain-specific operators (immune-inspired mechanisms, idle-slot reuse) to improve convergence under tight cost and time limits. Greedy-Ant [31], which uses Ant Colony Optimization (ACO) for DAG-based workflows, minimizes makespan using an ant colony system that iteratively reinforces promising task–VM mappings These swarm-intelligence methods are effective at exploring large search spaces but can incur high decision latency, making them less suitable for online scheduling.

Evolutionary Algorithms (EAs) are widely used for multi-objective workflow scheduling. NSGA-II [32] and NSGA-III [33] are the most popular frameworks, and several studies adapt them to cloud workflows. KMEWSA [34]

incorporates structural features of workflows to speed up convergence. EMWSA [35] and TETS [36] focus on energy-aware scheduling on DVFS-based resources, explicitly trading off makespan and energy. FBGA [37] introduces batch-based evolution for instance-intensive workflows in private clouds. In addition to dominance-based EAs, MOEA/D [38] decomposes the multi-objective scheduling problem into a set of scalar subproblems, which are optimized cooperatively. Although MOEA/D is not workflow-specific, its decomposition strategy makes it a strong candidate for many-objective workflow scheduling with numerous conflicting criteria. However, despite their flexibility, EA-based methods often suffer from slow convergence, high computational overhead, and limited ability to react to runtime dynamics, making them less suitable for real-time scheduling in large-scale cloud environments.

These metaheuristic studies demonstrate that it is possible to obtain diverse Pareto-approximate solution sets for cloud workflows, and many of them are evaluated using indicators such as Hypervolume and Inverted Generational Distance [17]. However, they generally assume offline or batch scheduling, rely on handcrafted solution representations, and suffer from high computational overhead in dynamic settings.

C. DEEP REINFORCEMENT LEARNING FOR CLOUD SCHEDULING

More recent work treats cloud scheduling as a sequential decision problem and applies Deep Reinforcement Learning (DRL). In workflow-oriented settings, DQTS [39] uses Deep Q-Learning to schedule DAG tasks with the dual objectives of makespan minimization and load balancing. The authors combine entropy-based reward fusion with experience replay and show that DRL can outperform heuristic baselines on synthetic workflows, though their Pareto analysis is restricted to only these two objectives and excludes energy considerations. DRL-Cloud [40] formulates workflow scheduling and VM provisioning as a semi-Markov Decision Process, aiming to reduce makespan, energy consumption, and task rejection rate under hard deadlines and dynamic electricity prices. LowCarbon [41] addresses the joint optimization of carbon emissions, energy cost, and QoS penalties through DRL, but does not include makespan or Pareto-based multi-objective balancing. DeepEnergyJSV2.0 [42] uses a PPO-based scheduler to minimize energy over hybrid workloads, focusing solely on a single objective. GNN-Flat [43] uses a GNN to model the workflow graph and a flat DRL framework to optimize makespan and energy. However, its single-level agent operates over a high-dimensional flat observation space, causing scalability and generalization issues as workflow size and structural complexity increase.

Other DRL studies focus on large-scale or hierarchical control. The hierarchical DRL framework in [44] employs a global agent for VM allocation and local LSTM-based agents for server-level power control, improving energy and latency, but it neither models workflow structures nor optimizes makespan. MARL [4] proposes a multi-agent actor-critic architecture for geo-distributed data centers powered by both renewable and non-renewable energy, showing benefits in reducing brown energy usage and makespan. HITS [45] introduces a two-level DRL scheduler that first selects a VM cluster and then chooses a VM using Double DQN, adapting its reward function based on cluster load to balance cost, overdue time, and load, yet it does not account for workflow-driven scheduling or energy consumption.

Several works, though not strictly workflow-specific, provide influential DRL schedulers for multi-resource clusters. DeepRM [46] and its extension DeepRM+ [47] learn job scheduling policies from image-like cluster states to minimize metrics such as average slowdown and turnaround time. ADRL [48] integrates anomaly detection with DRL to handle workload surges while preserving SLA adherence. Other variants consider privacy-aware training (DRL+FL [49]) or mobile edge offloading (IDRQN [50]).

While these works collectively demonstrate the growing potential of DRL for cloud and cluster scheduling, most DRL-based schedulers treat objectives via scalar reward functions (e.g., weighted sums), and a large fraction of the literature optimizes only a single primary objective such as makespan or energy. Consequently, Pareto-front evaluation with HV or IGD is rare; among DRL methods considered in Table 1, DQTS explicitly considers Pareto fronts but still extracts a single solution for deployment. This scalarisation-centric design can obscure trade-offs and makes it difficult to adapt policies when operator preferences change [51].

D. GNN-BASED SCHEDULING AND RESEARCH GAPS

Graph Neural Networks (GNNs) have recently emerged as a powerful tool for learning on structured data and have been successfully applied to scheduling problems in the FJSSP literature. Lei et al. [13] propose a multi-action GNN framework that directly reasons over job-machine graphs and outperforms traditional schedulers. In a follow-up work, Lei et al. [56] introduce a hierarchical approach for dynamic FJSSP, using GNNs at the lower level to handle random job arrivals. These studies demonstrate that GNNs can encode complex precedence constraints and machine flexibility more effectively than handcrafted features.

In cloud workflow scheduling, however, the integration of GNNs remains limited. GNN-Flat [43] is, to our knowledge, the only published study that combines GNNs with DRL for DAG-based cloud workflows. It uses a flat action space where the agent chooses task-VM pairs directly, and evaluations are conducted on synthetic workloads without real-world cloud traces. Moreover, the comparison against multi-objective baselines such as MOHEFT, NSGA-II, or MOEA/D is not comprehensive, and the approach is not explicitly designed to produce diverse Pareto-optimal solutions.

Across the broader DRL literature, few schedulers exploit the full DAG structure of workflows using graph encoders,

TABLE 1. Representative cloud scheduling studies (Wo = Workflow Oriented, Dyn = Dynamic, Onl = Online).

Algorithm	Category	Objectives	Wo	Dyn	Onl
MOILP [52]	Traditional	Makespan, Cost	Yes	No	No
HEFT [18]	List Scheduling	Makespan	Yes	No	No
MOHEFT [20]	List Scheduling	Makespan, Cost	Yes	No	No
MHRA [22]	List Scheduling	Makespan, Energy	Yes	Yes	Yes
CETSA [23]	Greedy Heuristic	Makespan, Energy, Cost	No	Yes	No
RMREC [24]	List Scheduling + Greedy	Energy, Budget	Yes	No	No
EPETS [53]	Reactive rule-based	Makespan, Energy, Deadlines	No	No	No
FERPTS [26]	Greedy + Negotiation-based	Runtime, Energy Cost	Yes	Yes	Yes
IMPSO [29]	Particle Swarm Optimization	Makespan, Cost, Deadline	Yes	No	No
Greedy-Ant [31]	Ant Colony Optimization	Makespan	Yes	No	No
EMWSA [35]	Evolutionary Algorithm	Makespan, Energy	Yes	No	No
FBGA [37]	Evolutionary Algorithm	Makespan	Yes	No	Yes
DQTS [39]	DRL	Makespan, Load Balance	Yes	Yes	Yes
DRLB-TSA [54]	DRL	Makespan, SLA, Energy	No	Yes	Yes
DeepRM [46]	DRL	Average Job Slowdown	No	Yes	Yes
DRL-Cloud [40]	DRL	Makespan, Energy, Task Rejection Rate	Yes	Yes	Yes
LowCarbon [41]	DRL	Energy, QoS	Yes	Yes	Yes
ODTS [55]	DRL	Energy	No	Yes	Yes
Liu et al. [44]	DRL	Energy, Latency	No	Yes	Yes
MARL [4]	DRL	Energy, Renewable Energy	Yes	Yes	Yes
DRL + FL [49]	DRL	Energy, Delay, Load Balance	No	Yes	Yes
HITS [45]	DRL	Cost, Overdue Time, Load Balance	No	Yes	Yes
GNN-Flat [43]	DRL	Makespan, Energy	Yes	Yes	Yes

and even fewer do so in a multi-objective, Pareto-aware manner. Most methods either ignore dependencies (e.g., independent task models) or flatten the workflow into queues, thereby losing structural information that could guide more informed decisions.

These observations highlight three main gaps that motivate our work. First, there is a lack of DRL-based schedulers that explicitly target multi-objective Pareto fronts for cloud workflows using indicators such as HV and IGD. Second, existing DRL methods rarely exploit DAG structure through GNNs, which limits their ability to generalize across diverse workflows. Third, prior GNN-based approaches rely on flat action spaces that scale poorly with the number of tasks and VMs. In contrast, our study combines a GNN encoder with a two-stage PPO-based DRL agent to decouple task selection from VM assignment. It explicitly explores trade-offs among makespan, energy, and QoS latency, and evaluates performance using Pareto-based metrics on both synthetic and real-world cloud traces.

III. METHODOLOGY

Fig. 3 presents the architectural overview of the DRL methodology adopted in the 2SD-GAT study. The process begins with the generation of synthetic datasets under various parametric configurations. A simulated cloud environment provides the foundation for training, where the DRL agent interacts with the environment by performing scheduling actions and receiving feedback in the form of rewards, updated states, and environmental observations. Through iterative learning, the agent aims to optimize multiple objectives: minimizing makespan, reducing energy consumption, and QoS-related penalties, the latter modeled as the cumulative latency of high-priority tasks.

The subsequent sections are structured as follows. The formal model formulation that underpins both dataset generation and the simulated environment is first presented. Lower-bound heuristics, which play a crucial role in reward formulation, are then introduced. This is followed by a discussion of the data preprocessing and feature extraction process. The reinforcement learning framework is then detailed, followed by the preference-based reward function designed for multi-objective optimization. Finally, the agent architecture and model design are elaborated, with particular emphasis on the integration of GNNs, specifically the Graph Attention Network (GAT).

A. PROBLEM MODELING

In this study, workflows are represented as DAGs, which are commonly used to model dependencies between tasks [40]. A DAG is an acyclic graph $G = (V, E)$, where V is the set of vertices (tasks) and $E \subseteq V \times V$ is the set of directed edges representing dependencies between tasks. For any directed edge $(u, v) \in E$, task u must be completed before task v can start.

Makespan is a key metric in scheduling workflows, representing the total time required to complete all tasks in a workflow. Given a DAG $G = (V, E)$ representing the workflow, the makespan M of the workflow is defined as in (1).

$$M = \max_{J_i \in V} t_{J_i}^{\text{end}} \tag{1}$$

Here, $t_{J_i}^{\text{end}}$ denotes the completion time of task $J_i \in V$. The value of $t_{J_i}^{\text{end}}$ can be derived as shown in (2).

$$t_{J_i}^{\text{end}} = t_{J_i}^{\text{start}} + T_{J_i,\text{VM}_k} \tag{2}$$

In this expression, $t_{J_i}^{\text{start}}$ denotes the time at which the scheduler assigns task J_i to virtual machine VM_k, and T_{J_i,VM_k} represents the execution time of task J_i on VM_k. For a scheduling decision to be valid, the task must be compatible with the selected VM. This compatibility is determined based on the resource requirements specified by the user — for

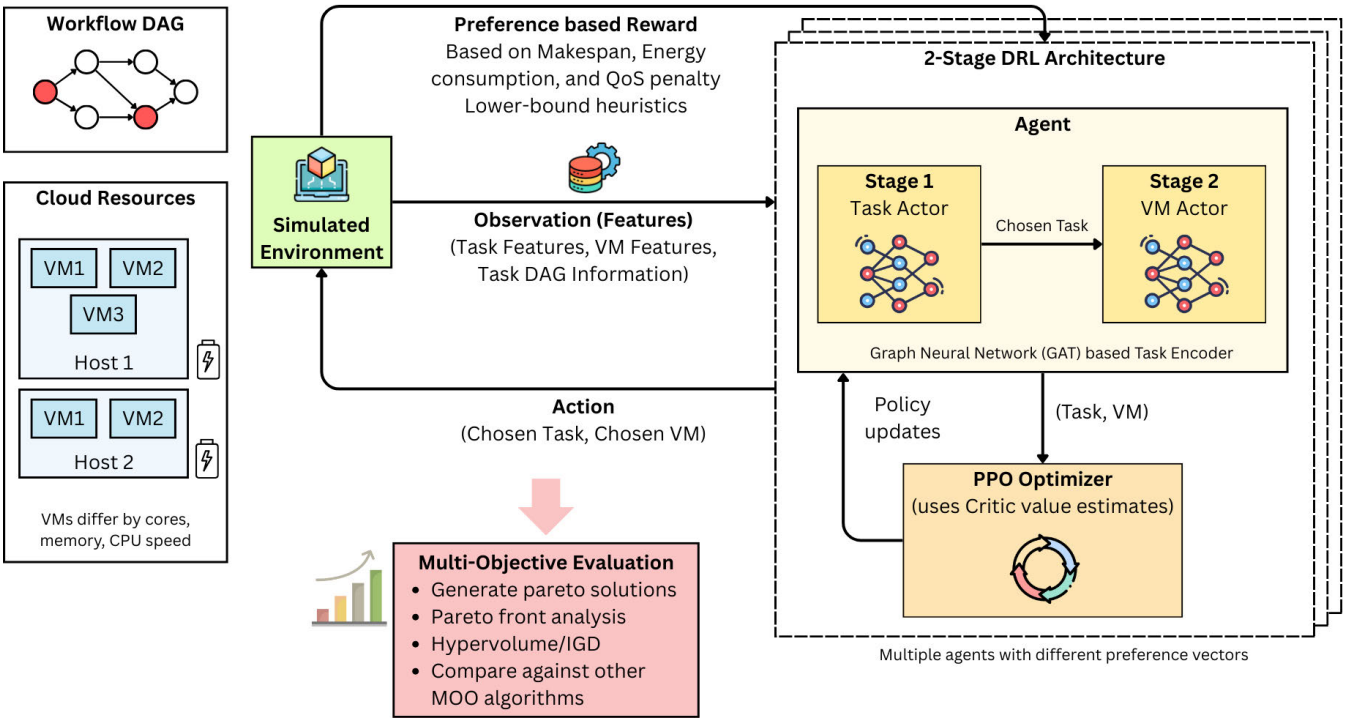


FIGURE 3. RL methodology used for the 2SD-GAT algorithm.

instance, a task may require a VM with certain memory capacity, number of cores, or other hardware characteristics. These constraints are treated as part of the user's resource requirements and must be strictly enforced by the scheduler to ensure service compliance. Hence, the assignment of tasks is restricted to VMs that satisfy the corresponding compatibility criteria.

In this study, compatibility between a task and a virtual machine is determined based on memory and core count requirements. A task J_i can be assigned to VM_j only if the following conditions are satisfied: $R_{J_i}^{\mathrm{mem}} \leq R_{\mathrm{VM}j}^{\mathrm{mem}}$ and $R_{J_i}^{\mathrm{core}} \leq R_{\mathrm{VM}j}^{\mathrm{core}}$, where $R_{J_i}^{\mathrm{mem}}$ and $R_{J_i}^{\mathrm{core}}$ represent the memory and core requirements of the task, respectively, and $R_{\mathrm{VM}j}^{\mathrm{mem}}$ and $R_{\mathrm{VM}j}^{\mathrm{core}}$ denote the memory and cores available on VM_j. When these conditions are met, the execution time T_{J_i,VM_j} can be computed using (3).

$$T_{J_i,\mathrm{VM}_j} = \frac{L_{J_i}}{S_{\mathrm{VM}_j}} \tag{3}$$

Here, L_{J_i} is the length of task J_i, and S_{VM_j} is the CPU speed of VM_j. It is assumed that the VM operates at full utilization with a constant CPU speed S_{VM_j} during the execution of the task.

Total energy consumption is a critical metric in cloud scheduling, representing the cumulative energy used by all hosts in a data center over a given period. The total energy consumption, denoted as E^{total}, is defined as in (4).

$$E^{\mathrm{total}} = \int_0^T P^{\mathrm{total}}(t)\, dt \tag{4}$$

Here, $P^{\mathrm{total}}(t)$ represents the total power consumption of the data center at time t. It is calculated as the sum of the power consumption of all N^H hosts, as shown in (5).

$$P^{\mathrm{total}}(t) = \sum_{i=1}^{N^H} P_{H_i}(t) \tag{5}$$

The power consumption $P_{H_i}(t)$ of host H_i at time t is determined by its utilization $U_{H_i}(t)$, which varies between the thresholds $U_{H_i}^{\mathrm{low}}$ and $U_{H_i}^{\mathrm{high}}$. The corresponding power values at these thresholds are $P_{H_i}^{\mathrm{low}}$ and $P_{H_i}^{\mathrm{high}}$. Assuming a linear relationship between utilization and power, $P_{H_i}(t)$ can be expressed as in (6).

$$P_{H_i}(t) = P_{H_i}^{\mathrm{low}} + (P_{H_i}^{\mathrm{high}} - P_{H_i}^{\mathrm{low}}) \cdot \frac{U_{H_i}(t) - U_{H_i}^{\mathrm{low}}}{U_{H_i}^{\mathrm{high}} - U_{H_i}^{\mathrm{low}}} \tag{6}$$

For simplicity, using the power at idle ($P_{H_i}^{\mathrm{idle}}$) and peak utilization ($P_{H_i}^{\mathrm{peak}}$), the above equation can be reduced to (7).

$$P_{H_i}(t) = P_{H_i}^{\mathrm{idle}} + (P_{H_i}^{\mathrm{peak}} - P_{H_i}^{\mathrm{idle}}) \cdot U_{H_i}(t) \tag{7}$$

The utilization $U_{H_i}(t)$ of host H_i is influenced by the VMs running tasks on it. Let $\mathrm{VM}_{H_i,t}^{\mathrm{active}}$ represent the active VMs on host H_i at time t. The utilization $U_{H_i}(t)$ is given by (8).

$$U_{H_i}(t) = \sum_{\mathrm{VM}_j \in \mathrm{VM}_{H_i,t}^{\mathrm{active}}} \frac{S_{\mathrm{VM}_j}}{S_{H_i}} \tag{8}$$

Here, S_{VM_j} is the CPU speed of VM_j, and S_{H_i} is the CPU speed of host H_i.

This study also incorporates task latency sensitivity through a priority-based model. For simplicity, two tiers of tasks were considered: low-priority and high-priority. The scheduler is expected to prioritize high-priority tasks during scheduling decisions. To quantitatively capture this behavior, we define the QoS penalty score (Q) as shown in (9).

$$Q = \sum_{J_i \in V} Q_{J_i} \tag{9}$$

Here, Q_{J_i} represents the QoS penalty score for task J_i, defined in (10).

$$Q_{J_i} = t_{J_i}^{\text{start}} \cdot R_{J_i}^{\text{priority}} \tag{10}$$

In this formulation, $t_{J_i}^{\text{start}}$ denotes the start time of task J_i, and $R_{J_i}^{\text{priority}} \in \{0, 1\}$ is a binary priority weight indicating whether the task is high-priority (1) or low-priority (0). This priority is assigned at the workflow level, meaning all tasks within a given workflow inherit the same priority as their parent workflow.

The objective is to find an optimal scheduling policy that minimizes makespan (M), total energy consumption (E^{total}), and QoS penalty (Q). This multi-objective optimization problem is formulated as shown in (11).

$$\text{minimize } \left(M, E^{\text{total}}, Q\right) \tag{11}$$

This optimization is subject to following constraints:

- Tasks must satisfy task precedence constraints defined by the DAG
- Tasks must satisfy memory requirements, $R_{J_i}^{\text{mem}} \leq R_{\text{VM}_j}^{\text{mem}}$
- Tasks must satisfy core count requirements, $R_{J_i}^{\text{core}} \leq R_{\text{VM}_j}^{\text{core}}$

B. LOWER BOUND HEURISTICS

In this study, lower-bound heuristics provide baseline estimations for makespan, energy consumption, and QoS penalty score, which are later used to formulate the reward function. These lower bounds serve as optimistic estimates, ensuring that the actual makespan, energy consumption, and the QoS penalty score will always be greater than or equal to these values.

The makespan lower-bound estimation at time t is calculated by determining the maximum estimated completion time among all tasks in the workflow DAG. For each task J_i, the completion time lower-bound depends on its scheduling status. If a task is scheduled, its actual completion time is used. For unscheduled tasks, the completion time lower-bound is computed by evaluating all compatible VMs and considering both parent task availability and the VM's readiness.

Formally, the makespan lower-bound estimation, denoted as M_t^{est}, is defined as shown in (12).

$$M_t^{\text{est}} = \max_{J_i \in V} M_{J_i,t}^{\text{est}} \tag{12}$$

Here, $M_{J_i,t}^{\text{est}}$ represents the estimated completion time lower bound of task J_i. This is recursively derived using (13).

$$M_{J_i,t}^{\text{est}} = \begin{cases} t_{J_i}^{\text{end}} & \text{if } J_i \text{ is scheduled,} \\ \min\limits_{\text{VM}_j \in \text{VM}_{J_i}^{\text{cmp}}} \left(\max\limits_{J_k \in J_{J_i}^{\text{par}}} \left(M_{J_k,t}^{\text{est}}, M_{\text{VM}_j,t}\right) + T_{J_i,\text{VM}_j} \right) & \text{otherwise.} \end{cases} \tag{13}$$

In this equation $\text{VM}_{J_i}^{\text{cmp}}$ represents the set of compatible VMs for task J_i, $J_{J_i}^{\text{par}}$ denotes the parent tasks of J_i, and $M_{\text{VM}_j,t}$ is the current finish time of VM_j, as defined in (14).

$$M_{\text{VM}_j,t} = \max_{J_k \in J_{\text{VM}_j,t}^{sch}} M_{J_k} \tag{14}$$

Here, $J_{\text{VM}_j,t}^{sch}$ refers to all the tasks scheduled on VM_j before time t, and M_{J_k} is the completion time of task J_k, as defined in (2).

The active energy consumption lower-bound estimation at time t is computed using the energy consumption explicitly caused by running jobs (disregarding the idle power consumption). If a task is scheduled, its active energy consumption is based on the task's execution time and the assigned VM. For unscheduled tasks, the active energy consumption is estimated as the minimum possible energy cost across compatible VMs.

Formally, the active energy consumption lower-bound estimation, denoted as E_t^{est}, is defined in (15).

$$E_t^{\text{est}} = \sum_{J_i \in V} E_{J_i,t}^{\text{est}} \tag{15}$$

Here, V represents all the tasks, and $E_{J_i,t}^{\text{est}}$ is the estimated active energy cost of task J_i. This cost is defined in (16).

$$E_{J_i,t}^{\text{est}} = \begin{cases} E_{J_i,\text{VM}_k}, & \text{if } J_i \text{ is scheduled to } \text{VM}_k, \\ \min\limits_{\text{VM}_j \in \text{VM}_{J_i}^{\text{cmp}}} \left(E_{J_i,\text{VM}_j}\right), & \text{otherwise.} \end{cases} \tag{16}$$

In this expression, E_{J_i,VM_j} is the active energy consumed when task J_i is scheduled on VM_j. It is calculated using (17).

$$E_{J_i,\text{VM}_j} = T_{J_i,\text{VM}_j} \cdot P_{\text{VM}_j} \tag{17}$$

Here, T_{J_i,VM_j} is the execution time of task J_i on VM_j, and P_{VM_j} is the power consumption of VM_j. The power consumption P_{VM_j} is derived as shown in (18) using (8) and the active component of (7).

$$P_{\text{VM}_j} = (P_{H_k}^{\text{peak}} - P_{H_k}^{\text{idle}}) \cdot \frac{S_{\text{VM}_j}}{S_{H_k}} \tag{18}$$

Here, $P_{H_k}^{\text{peak}}$ and $P_{H_k}^{\text{idle}}$ represent the peak and idle power consumption of the host H_k where VM_j is located. The term S_{VM_j} is the CPU speed of VM_j, and S_{H_k} is the CPU speed of the host H_k.

The QoS penalty score lower-bound estimation at time t is calculated by estimating the earliest possible start time for each task, weighted by its priority. For scheduled tasks, the start time is already known and used directly. For unscheduled tasks, the earliest feasible start time is computed by evaluating all compatible VMs and considering both parent task availability and the VM's readiness. This estimation provides a lower-bound on the cumulative QoS penalty score under ideal scheduling conditions.

Formally, the QoS penalty score lower-bound estimation, denoted as Q_t^{est}, is defined in (19).

$$Q_t^{\text{est}} = \sum_{J_i \in V} Q_{J_i,t}^{\text{est}} \tag{19}$$

Here, V denotes the set of all tasks, and $Q_{J_i,t}^{\text{est}}$ represents the estimated QoS penalty score of task J_i at time t, defined in (20).

$$Q_{J_i,t}^{\text{est}} = \begin{cases} 0 & \text{if } R_{J_i}^{\text{priority}} = 0, \\ t_{J_i}^{\text{start}} & \text{if } J_i \text{ is scheduled}, \\ \min\limits_{\text{VM}_j \in \text{VM}_{J_i}^{\text{cmp}}} \left(\max\limits_{J_k \in J_{J_i}^{\text{par}}} \left(M_{J_k,t}^{\text{est}}, M_{\text{VM}_j,t} \right) \right) & \text{otherwise.} \end{cases} \tag{20}$$

As in the makespan estimation, $\text{VM}_{J_i}^{\text{cmp}}$ is the set of compatible VMs for task J_i, $J_{J_i}^{\text{par}}$ denotes its parent tasks, and $M_{\text{VM}_j,t}$ is the current finish time of VM_j as defined in (14).

It is important to note that these lower bounds are used for reward normalization and shaping rather than as optimization targets themselves. Consequently, the learning process depends primarily on relative performance differences between actions, not on the absolute tightness of the bounds. Even when the estimates are loose or imperfect, they still provide a consistent optimistic reference scale that stabilizes reward magnitudes across workflows of different sizes and characteristics. In practice, this makes the DRL policy robust to moderate estimation errors, as suboptimal bounds may affect normalization scale but do not alter the underlying environment transitions or true objective evaluations.

C. DATA PREPROCESSING

To ensure faster convergence of the model, the input data is preprocessed into a structured format suitable for the reinforcement learning framework. The preprocessing step handles task-level, VM-level and the task dependency graph. Input feature information is given in table 2.

TABLE 2. Summary of input features.

Feature	Description
$F_{i,t}^1$ ($I_{J_i,t}^{\text{ready}}$)	Task schedulable - Binary flag indicating if task J_i is ready to be scheduled (i.e., all parent tasks are completed and J_i has not yet been scheduled).
$F_{i,t}^2$ ($I_{J_i,t}^{\text{sch}}$)	Task scheduled - Binary flag indicating whether task J_i is already scheduled.
$F_{i,t}^3$ ($M_{J_i,t}^{\text{est}}$)	Task completion time - Estimated completion time of task J_i if unscheduled, or actual completion time if already scheduled. (13)
$F_{i,t}^4$ ($E_{J_i,t}^{\text{est}}$)	Task energy consumption - Estimated or actual energy consumption of task J_i based on its assignment. (16)
$F_{i,t}^5$ ($Q_{J_i,t}^{\text{est}}$)	Task QoS penalty score - Estimated or actual QoS penalty score for task J_i, weighted by its priority. (20)
$F_{i,t}^6$ ($R_{J_i}^{\text{priority}}$)	Task priority - The QoS-priority level of task J_i (e.g., 0 or 1).
$F_{j,t}^7$ ($M_{\text{VM}_j,t}$)	VM completion time - Estimated time when VM j becomes available to execute a new task. (14)
$F_{i,j,t}^8$ ($I_{J_i,\text{VM}_j,t}^{\text{cmp}}$)	VM compatibility - Binary flag indicating whether VM j is compatible with task J_i, based on resource constraints.
$F_{i,j,t}^9$ ($T_{J_i,\text{VM}_j,t}$)	Execution time cost - Execution time required for task J_i on VM j, considering VM specs and task length.
$F_{i,j,t}^{10}$ ($E_{J_i,\text{VM}_j,t}$)	Energy consumption cost - Estimated energy consumed by task J_i if executed on VM j.
$A_{i,j,t}$ ($I_{J_i,J_j,t}^{\text{dep}}$)	Task dependency flag - Binary flag representing whether there is a dependency (directed edge) from task J_i to task J_j. If a task is scheduled to a VM, a new dependency is added between the previous task of the VM and the scheduled task.

D. REINFORCEMENT LEARNING FRAMEWORK

RL is a paradigm in machine learning in which an agent interacts with an environment to learn optimal actions through trial and error. The agent receives feedback in the form of rewards, which guide its learning to maximize cumulative rewards over time.

- State (s): Represents the environment's current condition. The state provides essential information for the agent to make decisions. Let S_t denote the state at time t.
- Action (a): An action is a choice made by the agent in a given state. Let A_t denote the action taken by the agent at time t.
- Reward (r): A scalar feedback signal provided by the environment after each action, indicating the immediate benefit of the action. The goal is to maximize the cumulative reward over time. Let R_{t+1} be the reward received after taking action A_t in state S_t.
- Policy (π): Defines the agent's behavior by mapping states to probabilities of selecting each possible action. The policy can be deterministic ($\pi(s) = a$) or stochastic ($\pi(a|s) = P(A_t = a|S_t = s)$).
- Value Function ($V(s)$): Estimates the expected cumulative reward an agent can obtain from a given state s, following a policy π. The value function is defined as in (21) [57] where $\gamma \in [0, 1)$ is the discount factor, which balances immediate and future rewards.

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \middle| S_t = s \right] \tag{21}$$

- Action-Value Function ($Q(s, a)$): Estimates the expected cumulative reward of taking an action a in state s and then following policy π. The action-value function is defined as in (22) [57].

$$Q^{\pi}(s, a) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^{k} R_{t+k+1} \middle| S_t = s, A_t = a\right] \quad (22)$$

In DRL literature, Proximal Policy Optimization (PPO) is a popular algorithm designed to improve the stability and efficiency of policy updates in continuous and discrete action spaces [58]. PPO optimizes a policy by iteratively updating it using a clipped surrogate objective, which prevents large policy updates and ensures stable learning. The objective function in PPO is defined as in (23) [58].

$$L^{\text{PPO}}(\theta) = \mathbb{E}_t\left[\min\left(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_t\right)\right] \quad (23)$$

In (23):

- $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$ is the probability ratio of the new policy π_θ to the old policy $\pi_{\theta_{\text{old}}}$.
- $\hat{A}_t$ is the advantage estimate at time t, calculated as $\hat{A}_t = Q(s_t, a_t) - V(s_t)$.
- ϵ is a hyperparameter that controls the range for clipping, limiting the extent of policy updates.

In complex scheduling environments such as cloud task scheduling, two-stage reinforcement learning can be used to decompose decision-making into multiple levels. A common two-stage architecture involves agents operating in sequence: the task agent selects the next task to schedule, while the VM agent selects the most suitable VM for executing the chosen task. This decoupled structure simplifies the action space, reduces combinatorial complexity, and allows for specialized learning at each level. Similar two-stage approaches have also been applied to the FJSSP, where one agent selects the next operation and another determines the machine to execute it, effectively handling task-machine assignment under complex constraints [13].

In this research, PPO is applied to the cloud workflow scheduling problem, where the agent's goal is to minimize several objectives. The reward R_t is designed to reflect the multi-objective optimization framework's focus, penalizing the agent for longer makespans, higher energy usage, and delayed tasks. To normalize the these reward segments, a normalization method is employed. The reward function is given in (24) where M_t^{est}, E_t^{est}, and Q_t^{est} are the makespan lower-bound, active energy consumption lower-bound, and QoS penalty score lower-bound estimations for the state S_t ((12), (15), and (19)). Parameters α, β, and γ balance the importance of minimizing makespan and energy consumption.

$$R_{t+1} = -\alpha\left(\frac{M_{t+1}^{\text{est}} - M_t^{\text{est}}}{M_{t+1}^{\text{est}}}\right) - \beta\left(\frac{E_{t+1}^{\text{est}} - E_t^{\text{est}}}{E_{t+1}^{\text{est}}}\right) - \gamma\left(\frac{Q_{t+1}^{\text{est}} - Q_t^{\text{est}}}{Q_{t+1}^{\text{est}}}\right) \quad (24)$$

The normalization factors α, β, and γ ensure that each objective contributes on a comparable scale, preventing any single objective from dominating the reward signal due to differences in magnitude. These factors are determined empirically using the method described in Section III-I.

It is important to note that the reward signal is computed from the change in the agent's estimated objective values before and after an action is taken. As shown in Eq. (24), the reward depends on the relative difference between the estimated makespan, energy, and QoS latency at time t and their updated estimates at time $t+1$. This formulation measures how much the agent's decision improves or worsens the expected outcomes, rather than relying on absolute objective magnitudes. Because the estimator is intentionally optimistic (actual values will always exceed these estimates), the agent is penalized whenever its action leads to a deterioration in the predicted objectives. This encourages consistently improving decisions and provides a more stable learning signal by normalizing objective changes and reducing variance across workflows of different scales.

E. PREFERENCE-BASED REWARD FUNCTION

To generate a Pareto front in multi-objective optimization, it is essential to obtain a diverse set of solutions that reflect different trade-offs between competing objectives. In this study, this is achieved by introducing a preference-based reward function. During training, each agent is assigned a set of preference weights representing its emphasis on specific objectives. This enables the same architecture to be trained under different objective priorities, producing a spectrum of solutions. When combined, these agents contribute to the construction of a Pareto front by spanning different regions of the objective space.

The reward function is defined as a weighted combination of changes in estimated makespan, energy consumption, and QoS penalty score, as shown in (25). The preference weights for each objective are denoted by λ_1, λ_2, and λ_3, and are fixed per agent during training. Mathematically, the scalar reward R_{t+1} is a linear combination of three normalized objective changes, weighted by the preference vector. This formulation allows the agent to learn a policy that optimizes a specific weighted combination of objectives, and by training multiple agents with different preference vectors, a diverse set of Pareto-optimal policies can be obtained.

$$R_{t+1} = -\lambda_1\alpha\left(\frac{M_{t+1}^{\text{est}} - M_t^{\text{est}}}{M_{t+1}^{\text{est}}}\right) - \lambda_2\beta\left(\frac{E_{t+1}^{\text{est}} - E_t^{\text{est}}}{E_{t+1}^{\text{est}}}\right) - \lambda_3\gamma\left(\frac{Q_{t+1}^{\text{est}} - Q_t^{\text{est}}}{Q_{t+1}^{\text{est}}}\right) \quad (25)$$

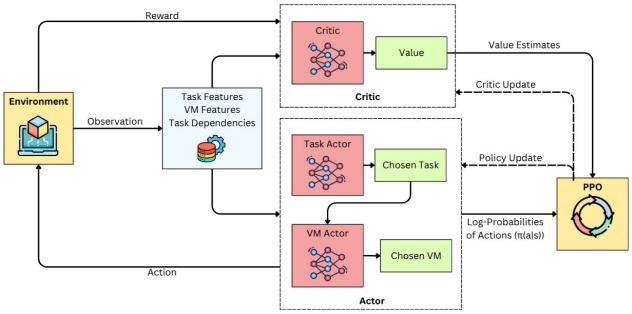


FIGURE 4. RL framework used for 2SD-GAT.

The preference weights λ_1, λ_2, and λ_3 are systematically enumerated using a uniform grid of the objective–weight space. Let $\boldsymbol{\lambda}^{(k)} = (\lambda_1^{(k)}, \lambda_2^{(k)}, \lambda_3^{(k)})$ denote the preference vector assigned to agent k. Each weight $\lambda_i^{(k)}$ is drawn from a discrete, uniformly spaced set over the interval [0, 1], defined as (26) where Δ denotes the grid resolution. Although alternative sampling strategies (e.g., simplex sampling or adaptive sampling) could have been used, the grid–search method was chosen for its simplicity and for providing explicit coverage of the objective–weight space.

$$\Lambda = \{\lambda \mid \lambda = n\Delta,\ n \in \mathbb{N},\ 0 \le \lambda \le 1\} \tag{26}$$

The complete set of candidate preference vectors is then constructed as the Cartesian product as in (27).

$$\mathcal{P} = \Lambda \times \Lambda \times \Lambda \tag{27}$$

To avoid redundant objective prioritizations, preference vectors that differ only by a positive scalar factor are removed, as they induce equivalent reward scaling and thus identical optimization behavior. Two preference vectors $\boldsymbol{\lambda}^{(i)}$ and $\boldsymbol{\lambda}^{(j)}$ are considered redundant if $\boldsymbol{\lambda}^{(i)} = c\,\boldsymbol{\lambda}^{(j)}, \quad c > 0$.

One limitation of the fixed preference approach is that it requires training multiple agents to cover the Pareto front comprehensively. An alternative approach would be to use a single agent with dynamic preference conditioning, where the preference vector is provided as an additional input to the policy network. This would allow a single agent to generate solutions for arbitrary preference vectors at inference time. However, this approach was not pursued in this study due to the increased complexity of the training procedure and the potential for reduced solution quality compared to dedicated agents.

The diversity and coverage of the Pareto front are evaluated using standard multi-objective metrics such as Hypervolume and Inverted Generational Distance, as discussed in the Section IV. These metrics provide quantitative evidence that the preference-based training strategy successfully generates a well-distributed set of Pareto-optimal solutions.

F. AGENT ARCHITECTURE AND MODEL DESIGN

To provide additional intuition into the agent's decision-making, the scheduling process can be viewed as a dynamic project management problem in which the workflow DAG evolves as tasks are scheduled and completed. At each decision step, the agent observes updated task states (e.g., ready and scheduled indicators), VM states, and the current dependency structure. These are encoded by the GAT to produce task embeddings that reflect both local properties and global structural roles, such as downstream depth and dependency influence. For instance, in a simple workflow where a preprocessing task must complete before two successor tasks can start, only the preprocessing task is initially marked ready. Once it finishes, the successor tasks become ready and the graph state is updated. The GAT then recomputes embeddings on the current DAG, allowing the policy to reassess which task has greater impact on future makespan, energy, and QoS objectives. The Task Actor selects a task accordingly, and the VM Actor assigns it to a suitable resource based on current VM conditions and objective trade-offs.

Unlike fixed heuristics, the learned policy is not a rigid rule-based procedure. It is optimized for long-term cumulative reward and may therefore choose actions that are not immediately optimal but lead to better overall scheduling performance over time.

Fig. 4 illustrates the overall data flow in the training architecture of 2SD-GAT. The environment exposes observations, including task features, VM features, and the workflow dependency graph, which are fed simultaneously to the Task Actor, VM Actor, and Critic. The Task Actor first selects the next task to schedule, which is then passed to the VM Actor to determine the most suitable virtual machine. The combined action, consisting of the selected task and VM, is applied to the environment, which in turn returns the next observation and a scalar reward signal. The Critic receives both the environment's observation and the corresponding reward, and computes value estimates used to derive advantage signals. These advantage estimates, along with the policy outputs from both actors, are passed to the PPO optimizer. PPO then updates the parameters of the Critic and both Actors [58].

1) STAGE 1: TASK ACTOR NETWORK

The Task Actor is responsible for selecting the next task J_i to be scheduled from the workflow DAG at each time step. Its input includes the task feature matrix $X^{\text{task}} \in \mathbb{R}^{N_t \times F_t}$, the VM feature tensor $X^{\text{vm}} \in \mathbb{R}^{N_t \times N_v \times F_v}$, and the task dependency graph $A \in \{0, 1\}^{N_t \times N_t}$, where N_t is the number of tasks, N_v is the number of VMs, and F_t, F_v are the dimensions of the task and VM features, respectively.

A core subcomponent of the Task Actor is the Graph Neural Network (GNN), implemented using a multi-layer Graph Attention Network (GAT). This GNN processes the workflow DAG to generate task embeddings that capture both local task properties and global structural dependencies. The same GAT-based task encoder is later reused by the VM Actor through the pooled task context p^{task}.

Each task J_i is represented as a node with an input feature vector $x_i \in \mathbb{R}^{F_t}$, formed by concatenating the task-level

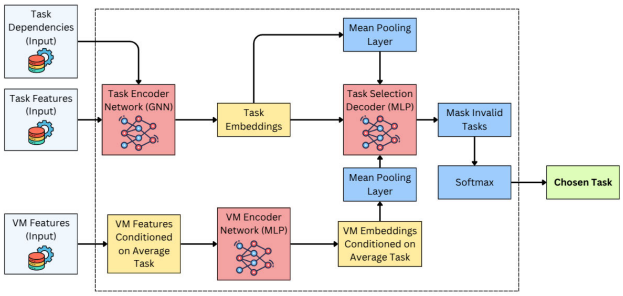


FIGURE 5. Task actor network architecture.

features $F_{i,t}^1$–$F_{i,t}^6$ listed in Table 2. These features encode the task's scheduling state and its estimated influence on the optimization objectives, providing the GNN with a rich descriptive basis for representation learning.

Task dependencies are expressed as directed edges in the workflow DAG. An edge (J_i, J_j) indicates that J_i must complete before J_j begins. These dependencies are stored in the adjacency matrix A, where $A_{i,j} = 1$ signifies a directed edge from J_i to J_j. Since assigning a task to a VM introduces a new precedence constraint with the previously executed task on that VM, the matrix is updated at each DRL time step t to incorporate both the original workflow edges and these dynamically introduced dependencies.

Although the edges do not carry explicit features beyond their binary connectivity, the GAT architecture leverages its attention mechanism to implicitly infer the importance of each dependency based on the connected node features. This is particularly beneficial in workflow scheduling, where some dependencies, such as those along or near the critical path, have disproportionately strong influence on makespan and overall performance. Unlike conventional GNNs using uniform aggregation weights, the GAT dynamically computes attention coefficients that determine how much information each neighbor contributes to a node's updated representation. This enables the model to emphasize structurally important relationships and to highlight tasks whose scheduling order is more critical.

The choice of GAT is further motivated by its strong generalization capabilities. Because attention weights are computed from both node features and learned embeddings, the encoder naturally adapts to workflow DAGs of varying depth, width, and structural complexity. The dynamic attention computation also evolves throughout training, allowing the GNN to refine how it integrates information from predecessors and successors as representations become more expressive. The effectiveness of this GAT-based encoder is validated through an ablation study presented in Section IV-C.

Moreover, the encoder's understanding of the evolving DAG is reinforced by key state-indicative inputs, particularly the task-ready ($F_{i,t}^1$) and task-scheduled ($F_{i,t}^2$) status features, together with the dependency edges ($A_{i,j,t}$) that are updated after every scheduling decision. These signals allow the GAT to continually reassess which nodes and precedence relations are most influential as tasks progress through different scheduling stages. At each DRL time step, the GAT operates on the current adjacency matrix $A_{i,j,t}$, thus, task embeddings are recomputed on the updated graph, ensuring that each decision is conditioned on the latest workflow topology. As a result, the network maintains an up-to-date structural view of the workflow, helping the agent preserve stable convergence even when newly introduced constraints modify the graph topology.

Let $x_i \in \mathbb{R}^{F_t}$ be the input feature vector of task J_i. In a GAT with L layers, the output embedding at layer l is computed by aggregating information from the neighbors $\mathcal{N}(i)$ using learned attention weights as defined in (28) [59].

$$h_i^{(l)} = \sigma\left(\sum_{j\in\mathcal{N}(i)} \alpha_{ij}^{(l)} W^{(l)} h_j^{(l-1)}\right), \quad h_j^{(0)} = x_j \tag{28}$$

The attention coefficients $\alpha_{ij}^{(l)}$ are computed using a shared attention mechanism that applies a LeakyReLU activation over concatenated linear projections of neighboring node embeddings [59].

After L layers, the final task embedding for each task J_i is denoted as $h_i^{\text{task}} = h_i^{(L)} \in \mathbb{R}^E$. These embeddings are aggregated via mean pooling to generate a global task-level context vector (task pool) $p^{\text{task}} \in \mathbb{R}^E$ using (29). The mean pooling layer aggregates task embeddings by computing their element-wise average across all tasks. Given N task embeddings, the pooled representation is obtained by averaging along the task dimension. This produces a global context representation while treating all tasks with equal importance. We adopt mean pooling due to its simplicity and stable aggregation behavior.

$$p^{\text{task}} = \frac{1}{N_t}\sum_{i=1}^{N_t} h_i^{\text{task}} \tag{29}$$

To incorporate information about VM availability, the average VM features across all tasks are computed for each VM. Let $X_{i,j}^{\text{vm}} \in \mathbb{R}^{F_v}$ denote the feature vector of VM j with respect to task i. The average VM feature representation $x_j^{\text{vm, avg}} \in \mathbb{R}^{F_v}$ is computed as in (30).

$$x_j^{\text{vm, avg}} = \frac{1}{N_t}\sum_{i=1}^{N_t} X_{i,j}^{\text{vm}}, \quad \forall j \in [1, N_v] \tag{30}$$

These features are passed through the VM encoder network, which is a MLP, to generate VM embeddings $h_j^{\text{vm, avg}} \in \mathbb{R}^E$. Each MLP layer consists of an affine transformation followed by a non-linear activation. The affine transformation at layer l is computed using (31), and the activation output is computed using (32).

$$z^{(l)} = W^{(l)}x + b^{(l)} \tag{31}$$

$$h^{(l)} = \text{ReLU}(z^{(l)}) \tag{32}$$

After passing through the MLP, the resulting VM embeddings are aggregated using mean pooling to form the

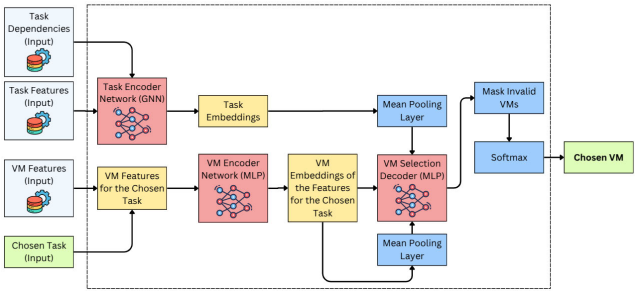


FIGURE 6. VM actor network architecture.

VM-level context vector (VM pool) $p^{\text{vm, avg}} \in \mathbb{R}^E$ as shown in (33).

$$p^{\text{vm, avg}} = \frac{1}{N_v} \sum_{j=1}^{N_v} h_j^{\text{vm, avg}} \tag{33}$$

For each task J_i, the task embedding h_i^{task} is concatenated with the global context vectors p^{task} and $p^{\text{vm, avg}}$ to form a combined representation e_i^{task} as shown in (34).

$$e_i^{\text{task}} = [h_i^{\text{task}} \parallel p^{\text{task}} \parallel p^{\text{vm, avg}}] \tag{34}$$

This vector is passed through a decoder MLP which outputs a scalar score s_i^{task} for each task. These scores are then masked using a binary mask that identifies schedulable tasks [13]. The masked score $\tilde{s}_i^{\text{task}}$ is defined in (35).

$$\tilde{s}_i^{\text{task}} = \begin{cases} s_i^{\text{task}} & \text{if } J_i \text{ is ready to be scheduled,} \\ -\infty & \text{otherwise.} \end{cases} \tag{35}$$

Finally, the task policy π^{task} is obtained using the softmax function over the masked scores as shown in (36).

$$\pi_i^{\text{task}} = \frac{\exp(\tilde{s}_i^{\text{task}})}{\sum_{j=1}^{N_t} \exp(\tilde{s}_j^{\text{task}})} \tag{36}$$

The selected task J_i is then sampled from this distribution during training or chosen using the maximum probability during evaluation.

2) STAGE 2: VM ACTOR NETWORK

The VM Actor is responsible for selecting an appropriate VM VM_j to execute the task J_i that was selected by the Task Actor. It uses the same VM encoder architecture (the MLP described in III-F1), but with a different choice of input features: since the task has already been chosen, the VM Actor only considers the corresponding VM feature slice $X_i^{\text{vm}} \in \mathbb{R}^{N_v \times F_v}$, which contains the VM features relevant to the selected task J_i.

These features are passed through the VM encoder network to generate VM embeddings $h_j^{\text{vm},i} \in \mathbb{R}^E$ for each VM j in the context of the chosen task i. The pooled VM-level context vector $p^{\text{vm},i}$ is computed via mean pooling over these embeddings as in (37).

$$p^{\text{vm},i} = \frac{1}{N_v} \sum_{j=1}^{N_v} h_j^{\text{vm},i} \tag{37}$$

To incorporate global context into the decision process, each VM embedding $h_j^{\text{vm},i}$ is concatenated with the task-level pooled vector p^{task} (computed by the GAT-based task encoder) and the VM-level pooled vector $p^{\text{vm},i}$ to form a combined representation e_j^{vm} for each VM as in (38).

$$e_j^{\text{vm}} = \left[h_j^{\text{vm},i} \parallel p^{\text{task}} \parallel p^{\text{vm},i}\right] \tag{38}$$

These combined vectors are passed through a decoder MLP to produce a scalar score s_j^{vm} for each VM, which represents the likelihood of selecting that VM for task J_i. To enforce task-VM compatibility constraints, these scores are masked using a binary mask. The masked scores are defined in (39).

$$\tilde{s}_j^{\text{vm}} = \begin{cases} s_j^{\text{vm}}, & \text{if VM is schedulable and compatible with } J_i, \\ -\infty, & \text{otherwise.} \end{cases} \tag{39}$$

The final VM selection policy, π_j^{vm}, is then computed by applying the softmax function over the masked scores similar to (36). During training, the VM is sampled from this probability distribution, while during evaluation, the VM with the highest probability is selected.

3) CRITIC NETWORK

The Critic network estimates the state value function $V(s)$, which represents the expected cumulative return starting from state s and following the current policy. This value estimate is used in PPO to compute the advantage function and guide policy updates.

The Critic receives the same observation as the actor, including task features, VM features, and the task dependency graph. The task features are encoded using the same GAT-based encoder as in the Task Actor to produce a pooled task representation $\mathbf{p}^{\text{task}}$. Similarly, the VM features are averaged across all tasks and encoded using a shared MLP encoder to obtain the pooled VM representation $\mathbf{p}^{\text{vm, avg}}$.

These two context vectors are concatenated to form the critic input embedding as defined in (40).

$$e^{\text{critic}} = \left[\mathbf{p}^{\text{task}} \parallel \mathbf{p}^{\text{vm, avg}}\right] \tag{40}$$

This combined embedding is passed through a MLP, which outputs a scalar value prediction $V(s)$ representing the expected return from the current state.

It is important to note that the Critic network does not share parameters with the Actor networks. While the Task Actor and VM Actor share their encoder and decoder parameters internally, the encoders used by the Critic are independently parameterized. This design allows the Critic to learn a stable and dedicated representation for value estimation without being influenced by the optimization dynamics of the policy network.

G. SYNTHETIC DATASET GENERATION

In each training or evaluation iteration, a new synthetic workflow dataset is generated based on a configurable set of

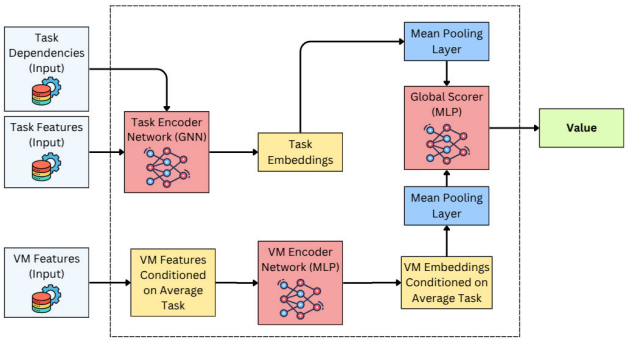


FIGURE 7. Critic network architecture.

TABLE 3. Host characteristics used for the dataset generation. Dataset taken from SPECpower_ssj® 2008 benchmark suite [62].

Name	Speed (GIPS)	Idle Power (W)	Peak Power (W)
PowerEdge R740	604.8	50	432
PowerEdge C6100	587.52	227	895
PowerEdge R820	332.8	110	446
iDataPlex dx360	187.712	116	475
PowerEdge R710	146.88	62	227

parameters that control DAG structure, task workloads, and host characteristics. This ensures that the agent is exposed to diverse workflow patterns and resource conditions, preventing overfitting to a fixed set of workflows. The following describes the methodology used to construct these synthetic datasets.

Each workflow is modeled as a DAG, generated using the Erdős-Rényi $G(n, p)$ method [60], where n denotes the number of nodes (tasks), and p is the probability of an edge existing between any two nodes. To ensure a high probability of connectivity and avoid isolated vertices, p is defined as $\frac{\log(n+\epsilon)}{n}$, where ϵ is a small positive constant [61]. The number of tasks n in each DAG is sampled uniformly from the range $[1, N_{\text{task}}^{\max}]$. A start node is added to each DAG to guarantee full connectivity. Each dataset is generated such that the total number of tasks across all workflows equals N_{task}.

The computational workload (length) of each task is sampled from a normal distribution centered at $\frac{L_{\text{task}}^{\min}+L_{\text{task}}^{\max}}{2}$, with a standard deviation of $\frac{L_{\text{task}}^{\max}-L_{\text{task}}^{\min}}{6}$. The sampled values are clipped to fall within the range $[L_{\text{task}}^{\min}, L_{\text{task}}^{\max}]$.

The number of VMs per workflow is randomly selected from $[1, N_{\text{vm}}^{\max}]$. The number of physical hosts is randomly chosen from $[1, N_{\text{host}}^{\max}]$, and each VM is randomly assigned to a host. VM memory and CPU core allocations are sampled uniformly from the range [0, 1]. Task resource requirements are also drawn from this range, with the constraint that each task remains compatible with at least one VM.

VM CPU speeds are sampled from the range $[S_{\text{vm}}^{\min}, S_{\text{vm}}^{\max}]$. Host-level characteristics, including idle and active power consumption, are obtained from the SPECpower benchmark suite [62]. The full configuration of host hardware characteristics is summarized in Table 3.

H. REAL-WORLD DATASET GENERATION

Similar to synthetic dataset generation, the real-world datasets are also produced using a configurable set of parameters. However, instead of sampling task and VM characteristics from predefined distributions, these parameters are derived directly from real-world statistics, ensuring that the generated characteristics reflect realistic distributions.

For task and VM characteristic generation, the Google cluster-usage traces v3 [63] dataset was utilized. This dataset provides operational data from eight different Borg cells for the month of May 2019. Given the large size of the dataset, analysis was limited to data from the America/New York timezone, specifically tasks that started after May 1, 2019 and before May 7, 2019. To ensure manageability, 100,000 records were randomly sampled to determine task and VM statistics. Additionally, only tasks with a runtime of less than 300 s were considered, as 300 s represents the maximum measurement length. Using these extracted statistics, the configurations for task lengths, resource requirements, and VMs were derived by applying appropriate scaling factors. This adjustment was necessary because many values in the Google trace are obfuscated for privacy reasons. The final configuration parameters used for real-world dataset generation are summarized in Table 4.

Although the real-world dataset parameters are derived from Google cluster-usage trace statistics, the datasets themselves are generated from aggregated summaries rather than direct trace replay. Consequently, fine-grained temporal behaviors such as burstiness, peak versus off-peak workload dynamics, and complex dependency structures present in the original traces may not be fully preserved. This design choice was made to enable the generation of a large number of randomized yet statistically grounded datasets for training and evaluation, ensuring controllability and reproducibility. A sensitivity analysis (IV-D) is included to verify that the learned policies respond consistently to changes in preference weights during training, providing additional evidence of robustness across configurations. Exploring full-fidelity trace replay and richer dependency modeling remains an avenue for future work.

The linear-branch-merge-sink DAG structure was employed to model task dependencies. Representative examples of such DAGs are illustrated in Fig. 8. This structure was selected for simulation as it reflects a foundational pattern commonly observed in production cloud workflows, including Extract-Transform-Load (ETL) processes, machine learning pipelines, and data orchestration systems. It effectively captures both sequential dependencies and parallel execution paths – two essential characteristics of real-world workflows. Moreover, this structure encompasses workflow patterns 1 (Sequence), 2 (Parallel Split), and 3 (Synchronization) as defined in [64].

Host characteristics were derived from the SPECpower benchmark suite [62], as described in Section III-G.

TABLE 4. Summary of configuration parameters derived from the Google cluster-usage traces for real-world dataset generation.

Description	Value(s)
VM CPU Speed (MIPS)	1188 (17%), 1473 (∼0%), 1818 (39%), 2178 (21%), 2946 (2%), 3072 (21%)
VM CPU Memory (MB)	683 (9%), 1024 (0%), 1366 (62%), 2048 (17%), 2732 (6%), 4096 (6%)
VM CPU Core Count	1 (22%), 2 (23%), 5 (26%), 10 (20%), 15 (9%), 25 (∼0%), 50 (∼0%)
Task Length Distribution	Normal Distribution $\mu = 3,732.02$ and $\sigma = 5,634.61$
Priority Task Percentage	68.39%

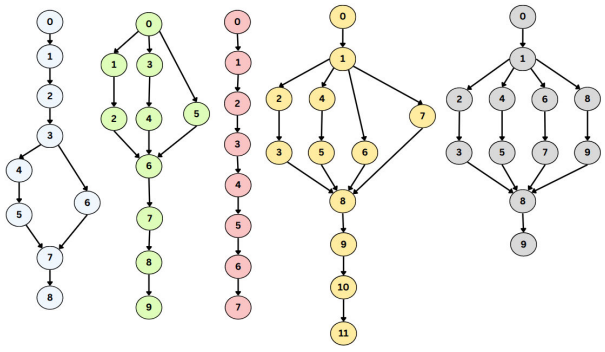


FIGURE 8. Linear-branch-merge-sink DAG structure.

To simulate realistic cloud environments, this study models two key aspects of dynamic behavior: random VM breakdowns and inaccurate task length estimation. Thus, a variant of the real-world dataset configurations was used with additional dynamic properties layered on top to mimic real operational uncertainty.

VM failures may occur during operation due to various causes such as system crashes, hardware malfunctions, or scheduled maintenance [65]. To account for this, parameters were introduced to control the frequency and timing of VM failures. For simplicity, only single-VM failures are modeled, ensuring that at any given moment, no more than one VM is marked as unavailable. To avoid deadlocks or unschedulable conditions, VMs that are the sole compatible option for any task are excluded from failure events.

In practical scenarios, task execution times are often estimated prior to scheduling, but these estimates may differ from actual runtimes due to variability in data, algorithmic complexity, or system-level fluctuations [6]. To reflect this, stochastic deviations between estimated and actual task lengths are incorporated, thereby capturing the uncertainty commonly encountered in runtime predictions.

I. TUNING REWARD FUNCTION PARAMETERS

The reward function weights α, β, and γ were tuned prior to training to ensure that no reward component was disproportionately emphasized. This tuning was performed by employing a random scheduling algorithm to drive the scheduler, and collecting the reward components across multiple samples. From these observations, inverse average values were computed and used as normalization factors, allowing each reward term to contribute approximately equally under random scheduling conditions.

J. SIMULATION ENVIRONMENT

The simulation for both training and testing was conducted using a custom-built Python simulation environment implemented using NumPy and PyTorch. The environment models workflow DAG execution, resource constraints, task scheduling, and energy consumption, and supports dynamic scenarios such as VM failures and task length uncertainty. The training and all the evaluations were conducted on a laptop equipped with an Intel Core i7-14650HX CPU, 16GB of RAM, and an NVIDIA RTX 4060 Laptop GPU with 8GB VRAM, running PyTorch 2.6 with CUDA 13.0 support.

K. AGENT TRAINING

1) AGENT TRAINING ON SYNTHETIC DATASET

The reinforcement learning agent was trained using the PPO algorithm, implemented in Python based on the CleanRL framework [66]. Weights were initialized using the default initialization schemes provided by PyTorch/PyTorch Geometric (Kaiming or Xavier depending on layer type). Training was performed over a fixed number of episodes, with each episode representing a complete scheduling scenario. In each episode, a new synthetic dataset was generated according to the parameters specified in Table 5. At the end of every episode, the agent's policy was updated using the PPO algorithm to maintain a balance between exploration and exploitation. The hyperparameters used for training are summarized in Table 6 and Table 7. For clarity and reproducibility, Table 6 also reports the random seed, number of parallel environments, and total training timesteps.

Fig. 9 illustrates the training curve of the agent with equal preference weights ($\lambda_1 = \lambda_2 = \lambda_3 = 1$), showing the increase in episodic return as training progressed. The training results indicate that the PPO algorithm, in conjunction with the GAT-based architecture, effectively learned to minimize the makespan and energy consumption while handling dynamic task arrivals and dependencies.

2) AGENT TRAINING ON REAL-WORLD DATASET

To demonstrate the benefits of fine-tuning and training the agent on real-world scenarios, additional training was performed using the real-world dataset described in Section III-H. The number of tasks (N_{task}), maximum number of tasks per workflow ($N_{\text{task}}^{\max}$), maximum number of VMs ($N_{\text{vm}}^{\max}$), and maximum number of hosts ($N_{\text{host}}^{\max}$) were kept as specified in Table 5. All training parameters as well were retained as specified in Table 6, except for the reward scaling factors α, β, and γ. These parameters were re-tuned using the method outlined in Section III-I, and their updated values are provided in Table 8.

Additionally, agents were trained under various preference configurations. Representative training curves for some of these agents are illustrated in Fig. 10.

TABLE 5. Training dataset generation parameters.

Parameter	Description	Value
N_{task}	Number of tasks	50
$N_{\text{task}}^{\max}$	Max number of tasks per workflow	10
$L_{\text{task}}^{\min}$	Min task length	500
$L_{\text{task}}^{\max}$	Max task length	100,000
$N_{\text{vm}}^{\max}$	Max number of VMs	5
$N_{\text{host}}^{\max}$	Max number of physical hosts	4
$S_{\text{vm}}^{\min}$	Min CPU speed (MIPS) for VMs	500
$S_{\text{vm}}^{\max}$	Max CPU speed (MIPS) for VMs	5000

TABLE 6. Summary of PPO hyperparameters used for training the agent and key reproducibility settings.

Parameter	Description	Value
lr	Learning rate (annealed)	2.5×10^{-4}
N	Batch size	512
γ	Discount factor	0.99
λ	Generalized Advantage Estimation (GAE) factor	0.95
ϵ	Clipping parameter	0.2
K	Number of epochs per PPO update	4
β	Entropy coefficient	0.01
c	Maximum gradient norm for clipping	0.5
B	Number of steps per policy rollout	128
M	Number of minibatches per update	4
θ_{te}	Task encoder (GAT) hidden layers	64 (3 layers)
θ_{ve}	VM encoder hidden layers	64 (2 layers)
θ_{d}	Task/VM decoder hidden layers	64 (2 layers)
θ_{gs}	Critic global scorer hidden layers	64 (2 layers)
E	Number of features in embeddings	8
	Random seed	0
	Total timesteps	200,000
	Early stopping criteria	None
	Number of parallel environments	4

TABLE 7. Reward scaling factors used for training the agent on the synthetic dataset.

Parameter	Description	Value
α	Weight of makespan reward	0.541
β	Weight of energy consumption reward	9.093
γ	Weight of QoS penalty score reward	0.425

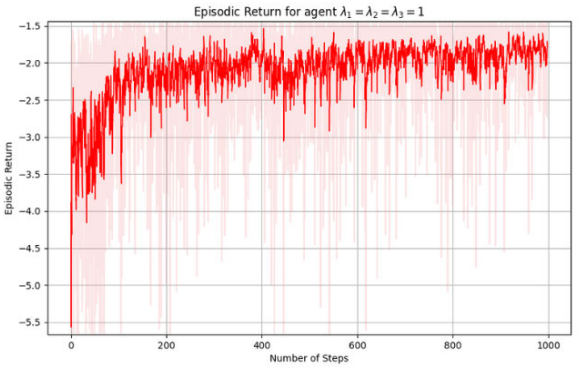


FIGURE 9. Episodic return during training of the agent with equal weights.

L. AGENT EVALUATION

For evaluating the RL agent, a synthetic dataset was generated with attributes given in table 9. During evaluation, the maximum number of workflows and tasks is varied to assess the model's performance across several scenarios.

In addition, evaluation was performed using a real-world dataset, denoted as DS^{real}. This dataset was derived from the method described in Section III-H. The parameters used to generate this dataset are summarized in Table 10.

An extended variant of this dataset, denoted as DS^{dyn}, was also used to simulate dynamic execution conditions, incorporating random VM breakdowns and inaccurate task length estimations. The specific parameters used to introduce these dynamic behaviors are listed in Table 11.

TABLE 8. Reward scaling factors used for training the agent on the real-world dataset.

Parameter	Description	Value
α	Weight of makespan reward	0.91
β	Weight of energy consumption reward	14.398
γ	Weight of QoS penalty score reward	0.602

TABLE 9. Evaluation synthetic dataset generation parameters.

Parameter	Description	DS_1^{syn}	DS_2^{syn}	DS_3^{syn}
N_{task}	Number of tasks	50	250	1000
$N_{\text{task}}^{\max}$	Max number of tasks per workflow	10	20	50
$L_{\text{task}}^{\min}$	Min task length	500	250	1
$L_{\text{task}}^{\max}$	Max task length	100,000	200,000	500,000
$N_{\text{vm}}^{\max}$	Max number of VMs	5	20	50
$N_{\text{host}}^{\max}$	Max number of physical hosts	4	5	10
$S_{\text{vm}}^{\min}$	Min CPU speed (MIPS) for VMs	500	250	100
$S_{\text{vm}}^{\max}$	Max CPU speed (MIPS) for VMs	5000	10,000	20,000

M. COMPARISON ALGORITHMS

2SD-GAT algorithm is evaluated against a diverse set of baseline algorithms to comprehensively assess its performance across multiple objectives and scheduling scenarios. Table 12 summarizes the comparison algorithms, which span both static and dynamic scheduling strategies, as well as single-objective and multi-objective optimization approaches.

Among the single-objective static algorithms, HEFT and FERPTS represent well-established heuristic methods that have been widely adopted in workflow scheduling due to their efficiency in minimizing makespan. Min-Min and Max-Min heuristics offer contrasting strategies by prioritizing tasks with the shortest and longest execution times, respectively. These methods serve as useful baselines for evaluating fundamental scheduling behavior.

For dynamic single-objective scheduling, algorithms such as Round-Robin, Least-Loaded-First, Random, and Weighted Dynamic Scheduling are included. These operate in an online manner and reflect practical strategies used in real-time cloud environments, particularly under uncertainty and varying workloads.

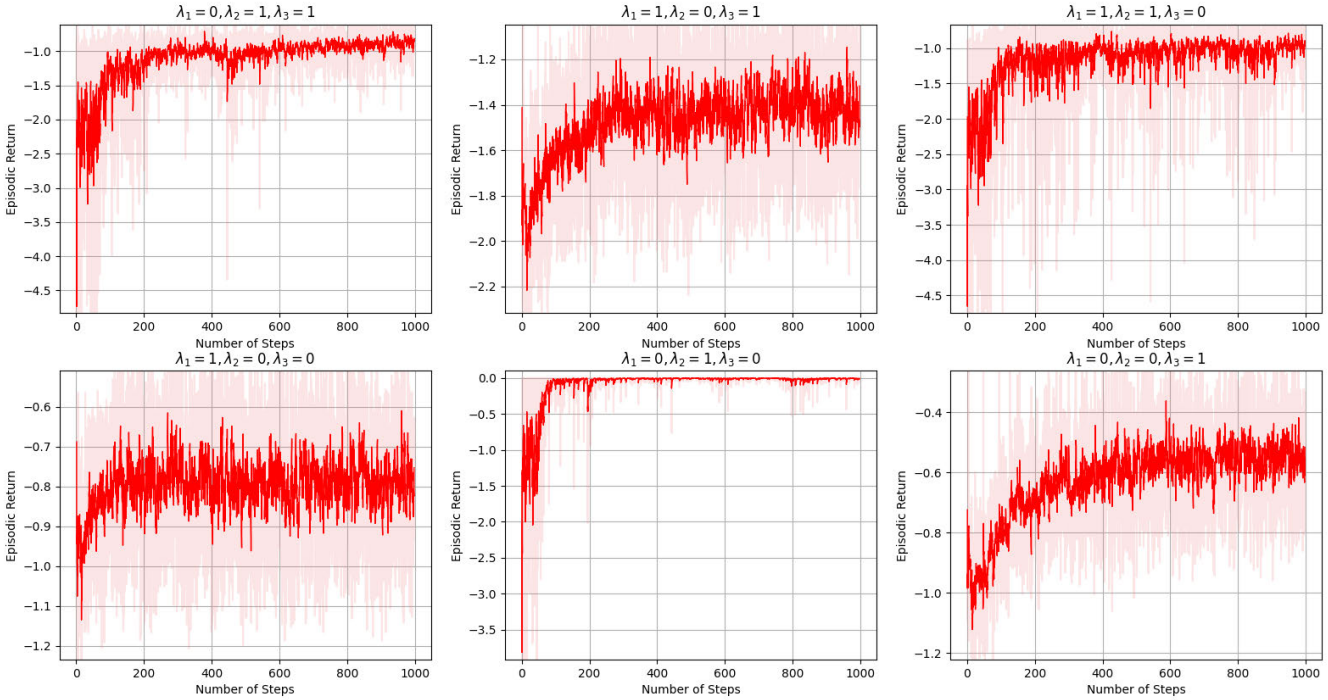


FIGURE 10. Episodic return during training of the some other agents.

To benchmark against state-of-the-art multi-objective optimization techniques, the evaluation includes MOHEFT, NSGA-II, NSGA-III, and MOEA/D. These evolutionary algorithms are designed to approximate the Pareto front across conflicting objectives such as makespan, energy consumption, and QoS penalties. They provide a strong comparative foundation to assess the effectiveness of 2SD-GAT in producing diverse and high-quality trade-offs.

In addition, GNN-Flat [43] is included as a learning-based baseline to provide a closer architectural comparison to 2SD-GAT. GNN-Flat was retrained using the same dataset and training budget to ensure a fair comparison. However, due to its simpler architecture and flat action space, it did not fully converge when optimizing all three objectives simultaneously within the given training budget (200,000 timesteps). Including GNN-Flat nonetheless offers insight into how architectural and action-space design choices influence multi-objective learning performance.

All algorithms were tested on the same datasets (DS_1^{syn}, DS_2^{syn}, DS_3^{syn}, DS^{real}, DS^{dyn}) under identical experimental conditions. The DRL methods (GNN-Flat and 2SD-GAT) were trained using the same datasets and simulation environment (Sections III-K1 and III-K2). To maintain fairness, the same input features and system-level information were provided to the baseline algorithms wherever applicable.

N. EVALUATION METRICS

The results of 2SD-GAT algorithm are compared with the baseline methods described in Section III-M. Due to the multi-objective nature of the problem, simple scalar comparisons of makespan, energy consumption, or QoS penalties are insufficient. Instead, standard metrics from MOO are employed to evaluate the performance of each algorithm.

One of the primary metrics used is Hypervolume, which measures the volume of the objective space dominated by the obtained Pareto front and bounded by a predefined reference point [16]. A higher hypervolume value indicates a better trade-off coverage among objectives. The reference point is chosen such that it is dominated by all solutions across all evaluated methods, ensuring fair comparison. The formal definition of HV is provided in (41).

$$HV(S, r) = \text{volume}\left(\bigcup_{x \in S} [f_1(x), r_1] \times \cdots \times [f_m(x), r_m]\right) \tag{41}$$

TABLE 10. Evaluation real-world dataset generation parameters.

Parameter	Description	Value
N_{task}	Number of tasks	250
$N_{\text{vm}}^{\max}$	Max number of VMs	20
$N_{\text{host}}^{\max}$	Max number of physical hosts	5

TABLE 11. Evaluation dynamic real-world dataset generation parameters.

Parameter	Description	Value
N_{task}	Number of tasks	250
$N_{\text{vm}}^{\max}$	Max number of VMs	20
$N_{\text{host}}^{\max}$	Max number of physical hosts	5
$T_{\text{downtime}}^{\max}$	Max VM downtime duration	20
$T_{\text{tbf}}^{\max}$	Max time between failures	40
$E_{\text{len}}^{\max}$	Max error rate of the task length estimation	0.1

TABLE 12. Summary of comparison algorithms.

Algorithm	Description
HEFT [18]	Heterogeneous Earliest Finish Time algorithm; prioritizes tasks and assigns them to VMs to minimize earliest finish times.
FERPTS [26]	Fast and Energy-Aware Resource Provisioning and Task Scheduling; a static energy-aware workflow scheduling method.
Random	Randomly assigns tasks to VMs at runtime.
Round Robin [67]	Assigns tasks to VMs in cyclic order regardless of task or VM characteristics.
Least-Loaded-First	Assigns tasks to the VM with the least current workload.
Weighted Dynamic	Dynamically assigns tasks to minimize a weighted score of all the objectives.
MOHEFT [20]	Multi-objective extension of HEFT producing Pareto-optimal schedules for multiple objectives.
NSGA-II [32]	Non-dominated Sorting Genetic Algorithm II; evolutionary algorithm for approximating Pareto fronts (Population size = 100, Generations = 500).
NSGA-III [33]	Improved variant of NSGA-II suitable for many-objective problems, using reference directions for diversity preservation (Population size = 100, Generations = 500, Ref Direction = Uniform 12 Partitions).
MOEA/D [38]	Multi-objective Evolutionary Algorithm based on Decomposition; decomposes a MOO problem into scalar sub-problems (Generations = 500, Ref Direction = Uniform 12 Partitions).
GNN-Flat [43]	A related graph-based approach with a flat action space and a different architecture, originally optimized for makespan and energy consumption, and retrained for makespan, energy consumption, and QoS objectives, under the same dataset and training budget as 2SD-GAT.

Here, S is the obtained Pareto front, r is the reference point, and $f_i(x)$ denotes the i-th objective value of solution x.

Another widely adopted metric is the Inverted Generational Distance, which evaluates both convergence and diversity of the Pareto front [16]. IGD calculates the average distance from each point in a known reference Pareto front to the closest point in the evaluated solution set. A lower IGD indicates that the solution set is closer to the true Pareto front. The IGD is defined in (42).

$$IGD(P^*, S) = \frac{1}{|P^*|} \sum_{x^* \in P^*} \min_{x \in S} \|x - x^*\| \tag{42}$$

Here, P^* is the reference Pareto front formed by aggregating all non-dominated solutions from all algorithms, and S is the solution set being evaluated.

In addition to these Pareto-based indicators, Decision Latency is also considered, which is the time taken to make a single scheduling decision. Since this study focuses on dynamic scheduling scenarios, it is critical that scheduling decisions are computed in real-time or near-real-time. Therefore, lower decision latency reflects better practical applicability of the method in real-world cloud systems.

Lastly, to analyze the dominance relationships between algorithms, the Pareto fronts produced by each method are compared. Given the three-objective setting, 2D projections of the Pareto front are presented for all three objective pairs to illustrate trade-offs and dominance across different dimensions.

IV. RESULTS AND ANALYSIS

A. RESULTS FOR SYNTHETIC DATASETS

This section presents a comprehensive evaluation of 2SD-GAT against a range of baseline methods under synthetic workload conditions. Table 13 reports the results for three synthetic datasets of increasing complexity. Corresponding visualizations are provided in Fig. 11 and 12, which display the Pareto fronts and multi-objective metric comparisons.

1) SMALL-SCALE WORKLOAD

As shown in Fig. 11, the Pareto front of 2SD-GAT overlaps significantly with several baselines. Given the relatively small number of tasks (50), the scheduling complexity is modest, allowing most algorithms to perform reasonably well. Nonetheless, 2SD-GAT is able to match or exceed the performance of both classical heuristics and multi-objective evolutionary algorithms.

From the bar charts and Table 13, it can be observed that 2SD-GAT achieves the highest hypervolume value ($(4.958 \pm 0.040) \times 10^{10}$), indicating that its Pareto front spans a larger and more optimal portion of the objective space. GAT-Flat exhibits high variance in its results, likely because it was originally specialized for optimizing energy consumption and makespan; extending it to include QoS may have reduced its training effectiveness under the same iteration budget as 2SD-GAT. Although NSGA-III achieves a slightly better IGD (164.818), its decision latency is significantly higher (1.060 s) compared to 2SD-GAT (0.070 s). This trade-off highlights the real-time efficiency of 2SD-GAT: it achieves near-optimal Pareto performance at a fraction of the computational cost.

2) MEDIUM-SCALE WORKLOAD

In Fig. 11, the Pareto front of 2SD-GAT shows a clear advantage over all baselines. Furthermore, the Pareto fronts produced by 2SD-GAT consistently lie in the same region, highlighting the stochastic stability of the approach. For the Makespan/Energy projection, the front covers a broad range of trade-offs, indicating that the method can flexibly adapt to different optimization preferences. On the other two projections, 2SD-GAT solutions dominate the majority of those produced by other schedulers.

In Table 13, 2SD-GAT achieves the highest hypervolume ($(6.605 \pm 0.105) \times 10^{12}$) and the lowest IGD ($4457.338 \pm 704.960$), outperforming all other approaches. These results indicate that the Pareto front of 2SD-GAT is not only widely spread but also lies close to the reference Pareto front. The decision latency remains exceptionally low (0.083 s), significantly outperforming evolutionary-based schedulers, such as NSGA-II (22.512 s), NSGA-III (21.954 s), and

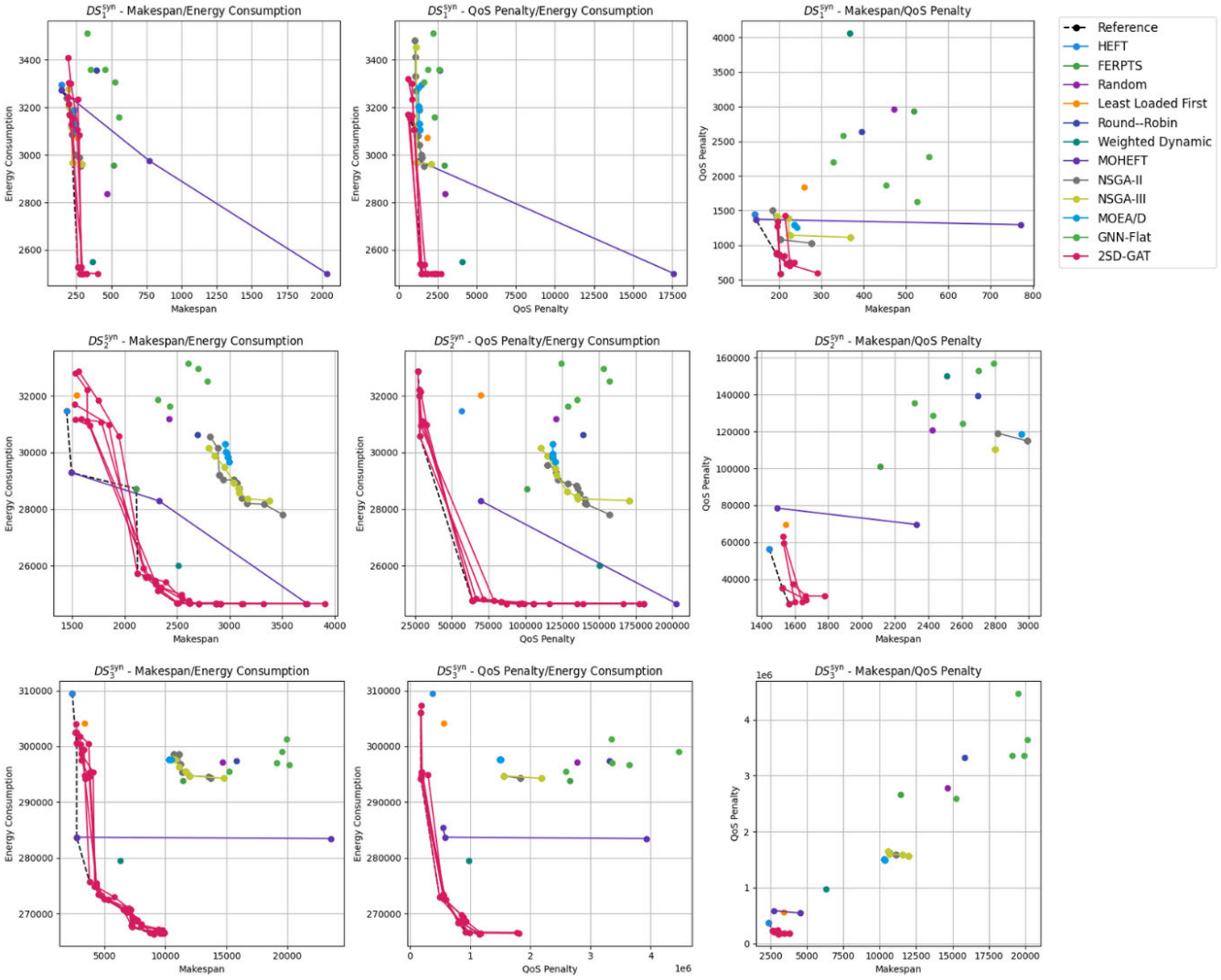


FIGURE 11. Pareto fronts of schedulers for synthetic datasets.

MOEA/D (20.210 s). This shows that 2SD-GAT scales effectively to larger workloads while preserving its practical deployability in dynamic environments.

3) LARGE-SCALE WORKLOAD

For the largest synthetic dataset (1000 tasks), the Pareto front generated by 2SD-GAT continues to outperform the baselines, demonstrating broad coverage and strong dominance across all objective pairings. This confirms that 2SD-GAT is scalable and generalizes well beyond the conditions on which it was trained.

According to Table 13, 2SD-GAT achieves the highest hypervolume ($(7.506 \pm 0.033) \times 10^{15}$) and the lowest IGD ($28790 \pm 3029$) among all methods, reflecting its ability to generate high-quality, diverse, and well-distributed Pareto-optimal solutions. Despite the computational complexity introduced by the large number of tasks, the decision latency remains just 0.244 s, a significant advantage over MOEA-based methods, which exhibit latencies exceeding 1000 s. At this scale, GNN-Flat exhibits substantial performance variance, likely due to incomplete convergence within the given training budget. This contrast further emphasizes the scalability of 2SD-GAT and reinforces that it is not only effective in generating optimal trade-offs but also viable for real-time use in large-scale, dynamic scheduling scenarios.

B. RESULTS FOR REAL AND DYNAMIC DATASETS

This section presents an evaluation of 2SD-GAT against a range of baseline methods under real-world workload conditions, both with and without dynamic disruptions. Table 14 reports the quantitative results for these two datasets. Corresponding visualizations are provided in Fig. 13 and 14, which illustrate the Pareto fronts and multi-objective metric comparisons. Two versions of the 2SD-GAT agent are evaluated: the agent trained on synthetic data, denoted as

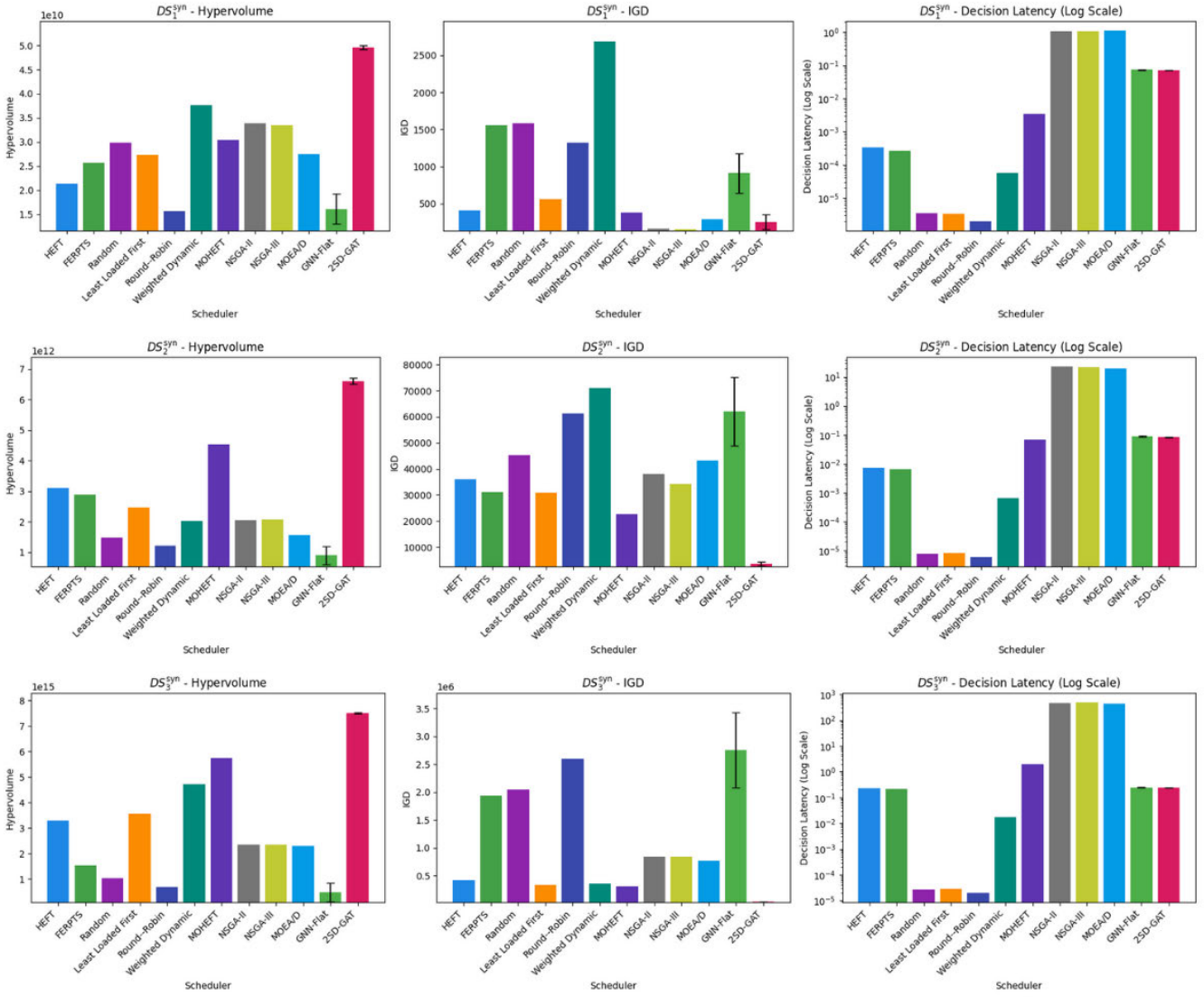


FIGURE 12. Hypervolume/IGD/decision latency of schedulers for synthetic datasets (Hypervolume - higher is better, IGD/Decision Latency - lower is better).

2SD-GAT (Syn), and the agent trained on real-world data without dynamic scenarios, denoted as *2SD-GAT (Real)*.

1) REAL-WORLD DATASET

In Fig. 13, the Pareto front generated by the 2SD-GAT (Real) model dominates the majority of solutions across all baseline methods, exhibiting strong spread and high-quality trade-offs across objective pairs. The 2SD-GAT (Syn) model, trained only on synthetic data, also performs competitively and dominates several classical and heuristic approaches. However, it shows a marginally lower performance compared to the 2SD-GAT (Real) model. This result is expected and likely stems from distribution shift. The synthetic training data differs in structure and workflow patterns, as it relies on Erdős-Rényi $G(n, p)$ graphs, while the real dataset consists of branch–parallel–merge DAGs that better reflect practical cloud workloads. Such differences naturally degrade IGD when a model is evaluated under domain shift rather than indicating strict dataset specificity.

These results validate the benefit of fine-tuning the RL agent on problem-specific data. The 2SD-GAT (Real) model, although trained on a small-scale real dataset (with 50 tasks), generalizes effectively to the 250-task workload seen in this evaluation, indicating that the architecture can adapt well to the target domain.

From Table 14 and Fig. 14, the 2SD-GAT (Real) method achieves the best hypervolume ($(1.678 \pm 0.014) \times 10^{10}$) and IGD ($2108.377 \pm 347.929$), indicating superior Pareto front quality. The 2SD-GAT (Syn) model follows closely, with the hypervolume ($(1.509 \pm 0.032) \times 10^{10}$) and second-best IGD ($1057.388 \pm 115.391$). Importantly, despite being trained on a different distribution, the synthetic-trained model remains the second-best overall, suggesting reasonable cross-domain generalization. Both models maintain decision latencies

TABLE 13. Performance comparison of 2SD-GAT and baselines for synthetic datasets (Best value is highlighted in bold text and second-best value is highlighted in underline).

Dataset	Algorithm	Best Makespan (s)	Best Energy Consumption (J)	Best QoS Penalty	Hypervolume	IGD	Decision Latency (s)
DS_1^{syn}	HEFT	**142.988**	3294.328	1448	2.140×10^{10}	405.837	0.000
	FERPTS	517.723	2955.773	2930	2.566×10^{10}	1560.023	0.000
	Random	471.080	2835.402	2961	2.980×10^{10}	1586.874	0.000
	Least Loaded First	260.105	3073.808	1836	2.739×10^{10}	560.872	0.000
	Round–Robin	395.300	3356.016	2639	1.566×10^{10}	1319.864	0.000
	Weighted Dyn	366.986	2549.781	4064	3.759×10^{10}	2688.466	0.000
	MOHEFT	144.988	**2500.936**	1298	3.039×10^{10}	378.735	0.003
	NSGA-II	184.335	2952.182	1031	3.385×10^{10}	164.818	1.060
	NSGA-III	195.616	2962.865	1115	3.341×10^{10}	**154.817**	1.060
	MOEA/D	236.616	3105.784	1258	2.745×10^{10}	290.348	1.067
	GNN-Flat	328.653	3158.429	1626.000	$(1.609 \pm 0.315) \times 10^{10}$	912.328 ± 269.623	0.073
	2SD-GAT (Proposed)	193.335	**2500.936**	**594.000**	$\mathbf{(4.958 \pm 0.040) \times 10^{10}}$	254.066 ± 98.246	0.070
DS_2^{syn}	HEFT	**1446.927**	31463.625	56536	3.098×10^{12}	44497.024	0.007
	FERPTS	2111.581	28724.176	101144	2.881×10^{12}	33611.087	0.006
	Random	2423.312	31192.025	120784	1.478×10^{12}	44976.566	0.000
	Least Loaded First	1544.312	32023.113	69641	2.468×10^{12}	38255.143	0.000
	Round–Robin	2696.646	30628.496	139205	1.207×10^{11}	57236.619	0.000
	Weighted Dyn	2510.855	26020.801	150227	2.016×10^{12}	65281.783	0.000
	MOHEFT	1494.927	**24670.243**	69712	4.526×10^{12}	22450.594	0.001
	NSGA-II	2814.312	27814.670	114909	2.052×10^{12}	35135.431	22.512
	NSGA-III	2800.312	28301.329	110477	2.065×10^{12}	31445.528	21.954
	MOEA/D	2958.312	29676.780	118479	1.565×10^{12}	43063.644	20.210
	GNN-Flat	2317.312	31623.227	124256	$(8.922 \pm 2.991) \times 10^{11}$	54660.148 ± 13167.484	0.087
	2SD-GAT (Proposed)	1524.773	**24670.243**	**26852**	$\mathbf{(6.605 \pm 0.105) \times 10^{12}}$	$\mathbf{4457.338 \pm 704.960}$	0.083
DS_3^{syn}	HEFT	**2374.953**	309527.515	378267	3.308×10^{15}	424177	0.228
	FERPTS	11429.292	293795.537	2657418	1.529×10^{15}	1933543	0.220
	Random	14679.624	297110.776	2775849	1.044×10^{14}	2052007	0.000
	Least Loaded First	3398.891	304178.976	561443	3.561×10^{15}	337402	0.000
	Round–Robin	15832.212	297348.726	3319678	6.951×10^{14}	2595804	0.000
	Weighted Dyn	6312.635	279493.576	978795	4.709×10^{15}	359445	0.017
	MOHEFT	2724.022	283468.285	548030	5.748×10^{15}	314396	1.950
	NSGA-II	10683.114	294262.963	1563610	2.343×10^{15}	840378	473.643
	NSGA-III	10566.920	294263.343	1563610	2.358×10^{15}	840378	478.179
	MOEA/D	10316.588	297620.965	1491165	2.290×10^{15}	770403	435.758
	GNN-Flat	15237.556	295475.133	2596706	$(4.771 \pm 3.781) \times 10^{14}$	2760824 ± 674788	0.247
	2SD-GAT (Proposed)	2613.353	**266421.366**	**178645**	$\mathbf{(7.506 \pm 0.033) \times 10^{15}}$	$\mathbf{28790 \pm 3029}$	0.244

well below 0.1 s, significantly outperforming evolutionary baselines like NSGA-II (21.223 s) and MOEA/D (20.093 s), reinforcing the practicality of the 2SD-GAT approach for real-time scheduling.

2) DYNAMIC REAL-WORLD DATASET

The dynamic variant of the real dataset introduces runtime uncertainties through simulated VM failures and inaccurate task length estimations. As shown in Fig. 13, the 2SD-GAT (Real) model continues to maintain strong performance, with a Pareto front that dominates most of the baselines and demonstrates a wide spread across all projections. The 2SD-GAT (Syn) model also performs well, reinforcing its robustness despite not being explicitly trained on dynamic scenarios.

In contrast, static scheduling algorithms struggle to adapt in the face of environmental changes, requiring frequent re-computation and rescheduling. This is reflected in their performance metrics: MOHEFT, NSGA-II, and MOEA/D exhibit long decision latencies ranging from 1 s to over 190 s. Their IGD scores also degrade, indicating poorer solution quality under dynamic conditions.

As reported in Table 14, the 2SD-GAT (Real) model achieves the best hypervolume ($(7.425 \pm 0.207) \times 10^8$) and IGD ($237.332 \pm 48.518$), demonstrating strong adaptability and robustness to environmental changes. The 2SD-GAT (Syn) model attains the second-best hypervolume ($(5.933 \pm 0.170) \times 10^8$) and a comparable IGD ($237.332 \pm 48.518$), indicating that even when trained on datasets with different statistical distributions and workflow structures, the model generalizes effectively. Although the fine-tuned 2SD-GAT (Real) outperforms 2SD-GAT (Syn), the performance gap remains modest. NSGA-II achieves the third-best hypervolume (5.855×10^8), suggesting that it generates a well-distributed set of solutions across the objective space. However, its substantially higher IGD (1066.093) indicates that these solutions lie farther from the true Pareto front, reflecting weaker convergence despite good diversity. Furthermore, its extreme decision latency (exceeding 3 minutes) limits its practicality for real-time or dynamic scheduling

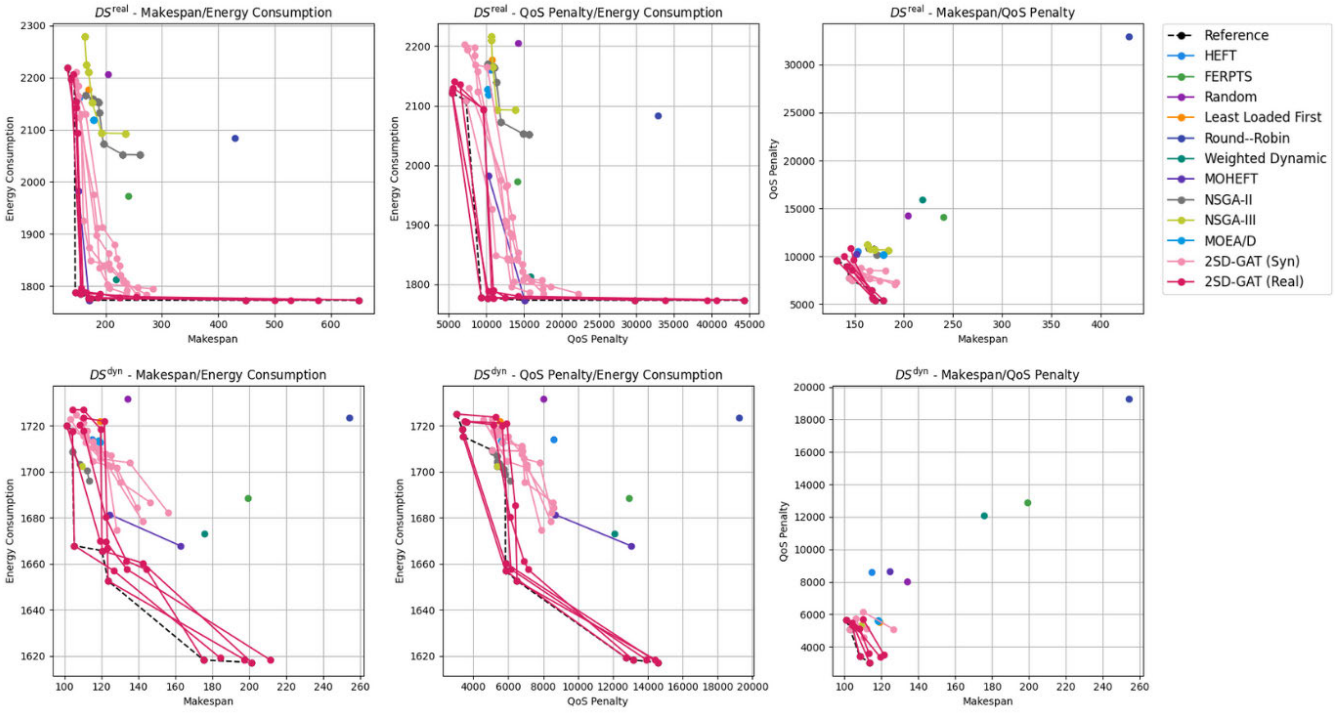


FIGURE 13. Pareto fronts of schedulers for real and dynamic datasets.

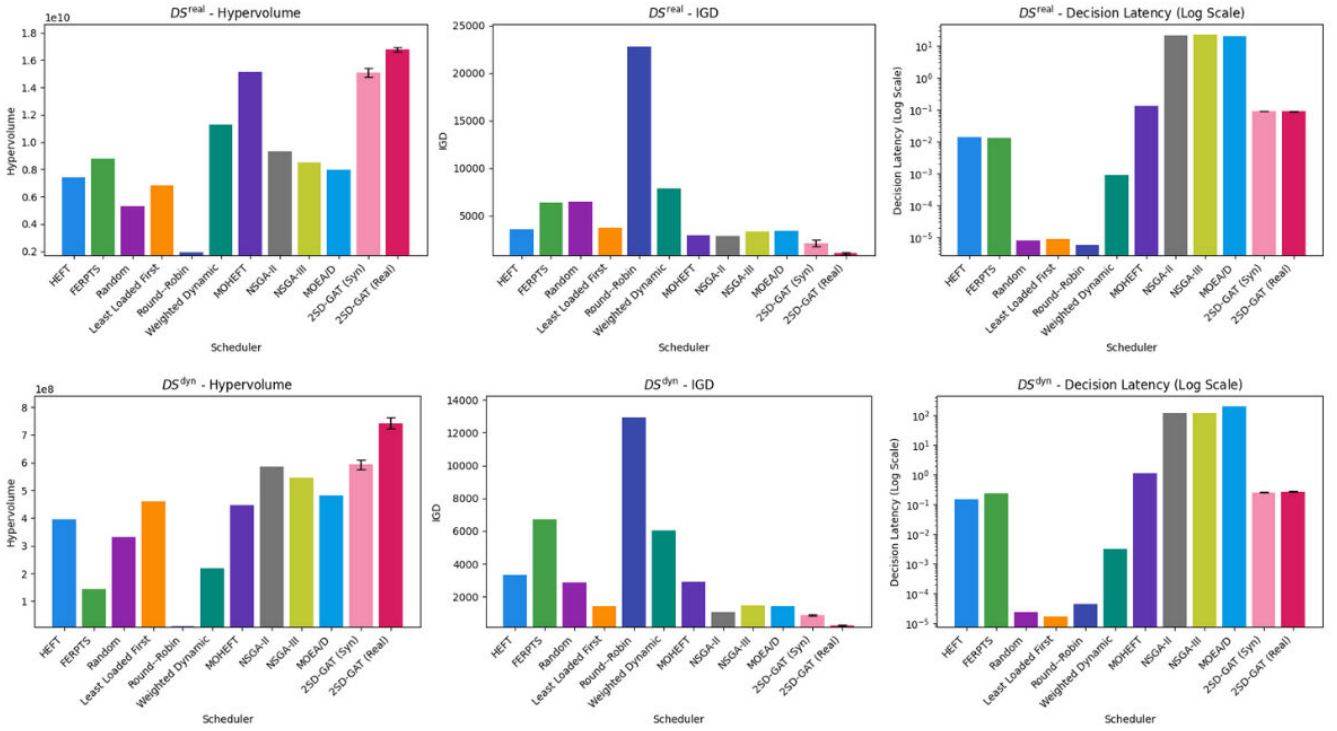


FIGURE 14. Hypervolume/IGD/decision latency of schedulers for real and dynamic datasets (Hypervolume - higher is better, IGD/Decision Latency - lower is better).

scenarios. In contrast, both 2SD-GAT (Syn) and 2SD-GAT (Real) outperform all other baselines in overall solution quality and decision latency, making them more suitable for practical deployment.

TABLE 14. Performance comparison of 2SD-GAT and baselines for real-world datasets (Best value is highlighted in bold text and second-best value is highlighted in underline).

Dataset	Algorithm	Best Makespan (s)	Best Energy Consumption (J)	Best QoS Penalty	Hypervolume	IGD	Decision Latency (s)
DS^{real}	HEFT	153.561	2160.173	10549	7.412×10^9	3598.126	0.013
	FERPTS	240.197	1972.795	14128	8.761×10^9	6356.601	0.013
	Random	204.197	2205.772	14236	5.289×10^9	6449.943	0.000
	Least Loaded First	169.248	2177.238	10798	6.805×10^9	3749.081	0.000
	Round–Robin	429.022	2084.045	32894	1.912×10^9	22836.320	0.000
	Weighted Dyn	218.939	1812.386	15916	1.129×10^{10}	7851.135	0.001
	MOHEFT	151.756	**1773.266**	10278	1.517×10^{10}	2962.564	0.131
	NSGA-II	164.197	2052.155	10184	9.352×10^9	2870.509	21.223
	NSGA-III	163.022	2092.714	10660	8.539×10^9	3353.743	21.780
	MOEA/D	179.197	2119.260	10194	7.984×10^9	3410.190	20.093
	2SD-GAT (Syn)	144.545	1783.226	7059	$(1.509 \pm 0.032) \times 10^{10}$	2108.377 ± 347.929	0.090
	2SD-GAT (Real)	**131.939**	**1773.266**	**5442**	$\mathbf{(1.678 \pm 0.014) \times 10^{10}}$	$\mathbf{1057.388 \pm 115.391}$	0.089
DS^{dyn}	HEFT	114.939	1713.977	8589	3.955×10^8	3320.501	0.148
	FERPTS	199.229	1688.687	12910	1.436×10^8	6727.567	0.237
	Random	133.965	1731.580	8029	3.316×10^8	2880.840	0.000
	Least Loaded First	119.229	1721.968	5510	4.593×10^8	1433.281	0.000
	Round–Robin	254.229	1723.616	19250	8.864×10^6	12930.079	0.000
	Weighted Dyn	175.626	1673.257	12077	2.191×10^8	6061.232	0.003
	MOHEFT	124.471	1667.807	8670	4.462×10^8	2935.953	1.131
	NSGA-II	104.229	1696.270	5096	5.855×10^8	1066.093	116.829
	NSGA-III	109.229	1702.322	5351	5.459×10^8	1478.809	117.356
	MOEA/D	118.229	1712.737	5623	4.820×10^8	1408.739	194.476
	2SD-GAT (Syn)	103.229	1674.673	4572	$(5.933 \pm 0.170) \times 10^8$	896.610 ± 41.346	0.254
	2SD-GAT (Real)	**101.229**	**1617.236**	**3045**	$\mathbf{(7.425 \pm 0.207) \times 10^8}$	$\mathbf{237.332 \pm 48.518}$	0.272

C. ABLATION STUDY

To assess the contribution of the GAT-based encoder within the proposed 2SD-GAT framework, an ablation study was conducted by replacing the GAT component of the Task Actor with two alternative architectures: a Graph Isomorphism Network (GIN) [68] and a multilayer perceptron (MLP). These variants, denoted as 2SD-GIN and 2SD-MLP respectively, preserve all other components of the two-stage scheduling architecture. This ensures that any performance differences arise solely from the choice of graph encoder used to process workflow dependencies.

The GIN baseline represents a powerful message-passing GNN with injective aggregation, enabling it to distinguish structurally different neighborhoods. However, unlike GAT, it applies uniform weights to all neighbors during aggregation and therefore cannot modulate the relative importance of different dependencies. The MLP baseline removes explicit graph modeling entirely, operating only on per-task feature vectors and relying on the DRL agent to learn implicit structural cues without a dedicated message-passing mechanism. Both alternatives constitute meaningful points of comparison for isolating the benefits of attention-based aggregation.

All three architectures, 2SD-GAT, 2SD-GIN, and 2SD-MLP, were evaluated using the DS^{real} real-world dataset configuration described in Section III-H, ensuring that each model encounters identical workflow structures, task characteristics, and VM configurations during evaluation. The detailed results of this ablation study are reported in Table 15, with Fig. 15 providing a visual comparison.

The quantitative results reveal several key insights. First, while 2SD-MLP achieves competitive performance in certain single-objective metrics such as best QoS penalty (5502), it consistently falls behind 2SD-GAT in MO performance. Specifically, 2SD-GAT achieves the highest hypervolume ($(4.125 \pm 0.031) \times 10^{10}$) and the lowest IGD ($306.060 \pm 51.785$), indicating substantially better Pareto-front coverage and convergence. In contrast, 2SD-MLP, despite being the second-best performer, attains a noticeably weaker IGD (954.055 ± 157.106) and a lower hypervolume ($(4.092 \pm 0.036) \times 10^{10}$).

The 2SD-GIN variant performs noticeably worse than both 2SD-GAT and 2SD-MLP across all MO metrics, with the lowest hypervolume ($(4.005 \pm 0.055) \times 10^{10}$), the highest IGD ($1184 \pm 473.192$), and the worst QoS penalty (7675). This degradation is consistent with the architectural limitations of GIN: while it preserves expressive neighborhood aggregation, its uniform weighting prevents it from differentiating between more and less critical dependencies, an ability that is particularly important in workflow DAGs with heterogeneous structural bottlenecks.

All approaches exhibit nearly identical average decision latencies (0.083, 0.087, and 0.085), indicating that the added expressiveness of attention-based aggregation does not introduce measurable runtime overhead. In practice, this shows that 2SD-GAT maintains competitive inference efficiency while delivering substantially higher solution quality.

Overall, the ablation findings reinforce the suitability of GAT as the encoder within the two-stage DRL architecture.

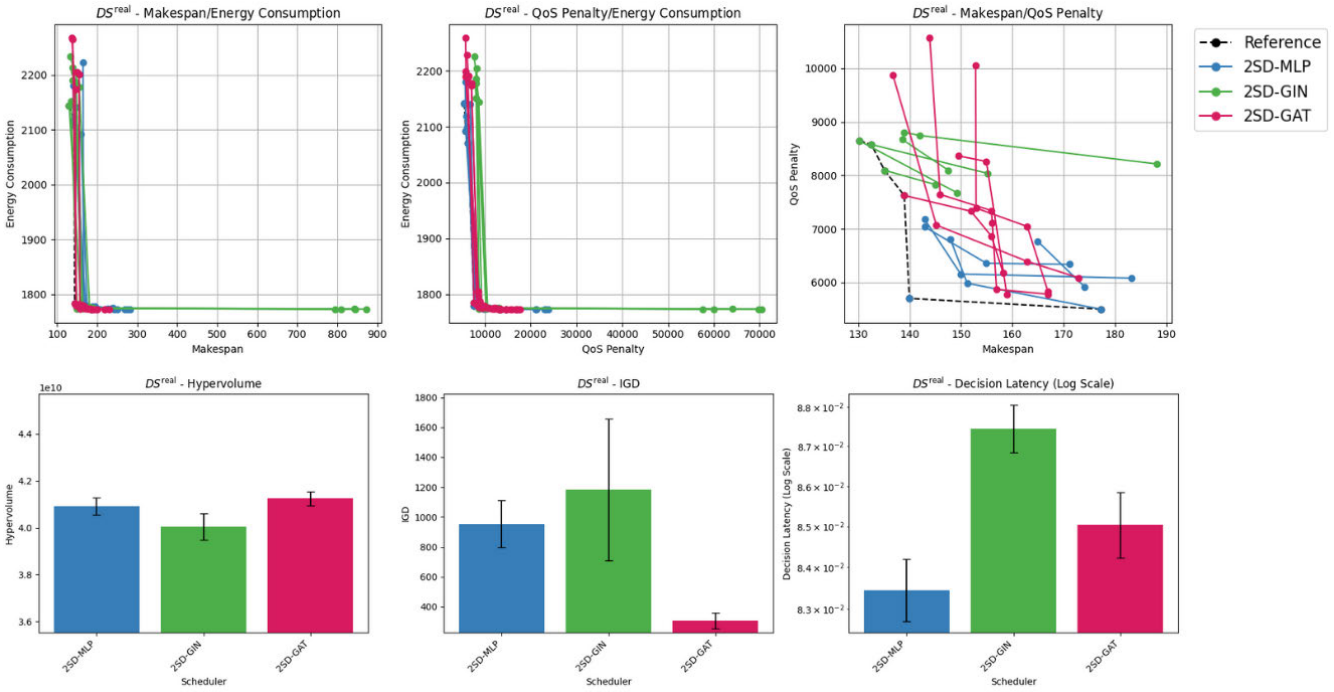


FIGURE 15. Pareto fronts/hypervolume/IGD/decision latency of ablation study (Hypervolume - higher is better, IGD/Decision Latency - lower is better).

TABLE 15. Performance comparison of 2SD-GAT, 2SD-GIN, and 2SD-MLP schedulers (Best value is highlighted in bold text and second-best value is highlighted in underline).

Dataset	Algorithm	Best Makespan (s)	Best Energy Consumption (J)	Best QoS Penalty	Hypervolume	IGD	Decision Latency (s)
DS^{real}	2SD-MLP	139.939	**1773.266**	**5502**	$(4.092 \pm 0.036) \times 10^{10}$	954.055 ± 157.106	**0.083**
	2SD-GIN	**130.197**	**1773.266**	7675	$(4.005 \pm 0.055) \times 10^{10}$	1184.586 ± 473.192	0.087
	2SD-GAT	136.721	**1773.266**	5777	$\mathbf{(4.125 \pm 0.031) \times 10^{10}}$	**306.060 ± 51.785**	0.085

D. SENSITIVITY ANALYSIS

In preference-based multi-objective reinforcement learning, it is important to verify that the learned policies respond consistently to changes in the preference weights used during training. Since each agent in the proposed framework is trained using a fixed preference vector $(\lambda_1, \lambda_2, \lambda_3)$, the expected behaviour is that increasing the weight assigned to a particular objective should steer the learned policy toward performance outcomes that reflect that objective's preference. To assess whether this holds in practice, a post-hoc sensitivity analysis was conducted on the trained agents.

The analysis considers the mapping from preference weights to the resulting performance metrics: makespan, energy consumption, and QoS penalty. A total of 26 agents trained under different preference tuples were evaluated on the same test workflows, producing paired data of the form $(\lambda_1, \lambda_2, \lambda_3) \mapsto (M, E, Q)$. To quantify how strongly each objective responds to its corresponding preference weight, we fit a separate linear regression model for each performance metric, using the preference weights as independent variables. This yields a first-order approximation as given in (43):

$$y = \beta_0 + \beta_M \lambda_1 + \beta_E \lambda_2 + \beta_Q \lambda_3, \tag{43}$$

The resulting regression coefficients are reported in Table 16, with Fig. 16 providing a visual interpretation of the linear trends.

Quantitatively, the makespan model shows that λ_1 exerts a negative influence ($\beta_M = -6.89$), confirming that stronger makespan preference consistently drives the policy toward faster schedule completion. In contrast, the positive coefficient for λ_2 ($\beta_E = 17.31$) reveals that energy-aware agents tend to produce longer schedules, a natural consequence of adopting more conservative or energy-efficient allocation strategies.

Energy consumption exhibits the strongest and most interpretable sensitivity patterns. The large negative coefficient associated with λ_2 ($\beta_E = -40.60$) shows that agents prioritising energy substantially reduce resource usage. Meanwhile, the smaller positive coefficients for λ_1 and λ_3 (1.97 and 1.44, respectively) indicate that agents focusing on

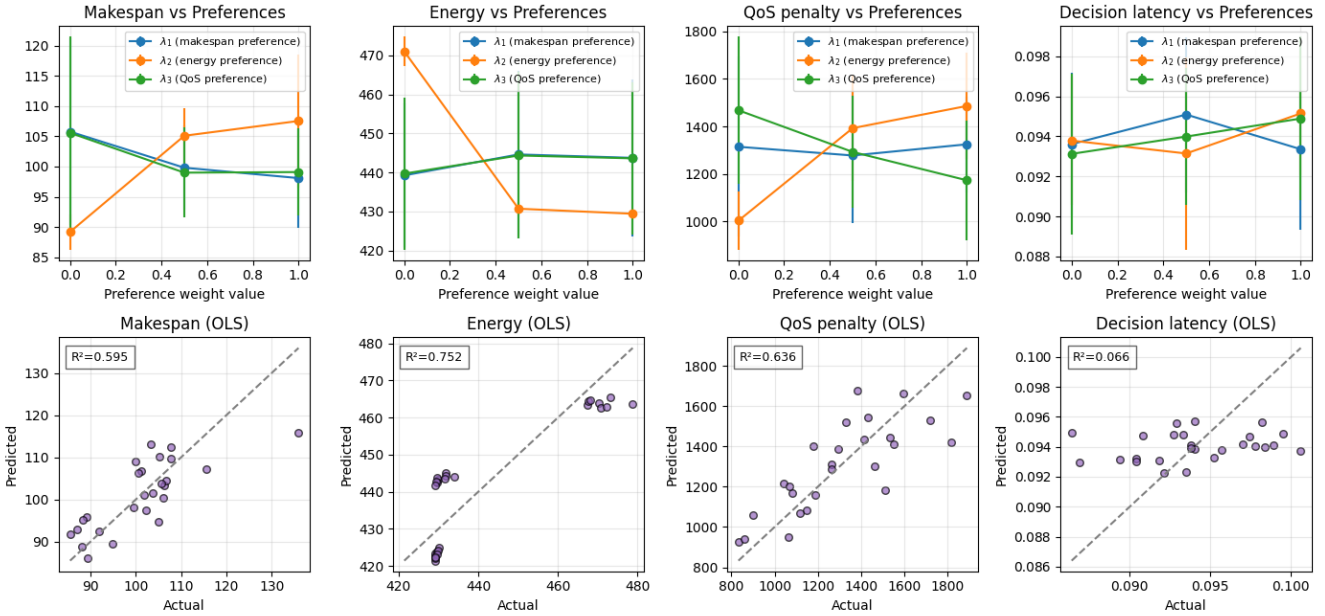


FIGURE 16. Sensitivity of agent performance to preference weights (Top) and sensitivity analysis of agent behaviour under varying preference weights (bottom).

TABLE 16. Linear sensitivity models relating preference weights ($\lambda_1, \lambda_2, \lambda_3$) to performance metrics.

Metric	β_0	β_M	β_E	β_Q	R^2
Makespan	98.6015	−6.8887	17.3124	−5.7111	0.5955
Energy Consumption	462.0045	1.9710	−40.6047	1.4411	0.7522
QoS Penalty	1189.9709	24.1977	461.7810	−263.6079	0.6358
Decision Latency	0.0923	−0.0001	0.0015	0.0019	0.0656

makespan or QoS tend to use slightly more resources. The high coefficient of determination ($R^2 = 0.75$) suggests that the preference weights explain most of the variation in energy usage.

QoS penalty responds strongly to both λ_2 and λ_3. The large negative coefficient for λ_3 ($\beta_Q = -263.61$) shows that increasing the QoS preference dramatically reduces QoS penalties, while the positive coefficient for λ_2 ($\beta_E = 461.78$) indicates that energy-efficient scheduling significantly increases QoS penalties. This trade-off aligns with the expected behaviour of agents operating under resource-saving constraints.

Decision latency exhibits negligible sensitivity to the preference weights, as reflected by the extremely small coefficients (all on the order of 10^{-3} or smaller) and the low R^2 value (0.0656). This indicates that the choice of preferences does not materially affect the time required for the agent to compute scheduling decisions, suggesting that latency is dominated by architectural factors rather than behavioural ones.

Overall, the sensitivity analysis demonstrates that the 2SD-GAT framework responds in a stable and interpretable manner to changes in the preference vector.

E. COMPUTATIONAL COST AND TRAINING TIME

All experiments were conducted on the hardware described in Section III-J. Each agent was trained for 200,000 environment steps. Across all trained preference-conditioned agents, the average training time per agent for the DS^{real} was 16.07 ± 1.18 minutes. Since the Pareto front is constructed using 26 agents with different preference vectors, the total sequential training budget is about 7 hours. This cost scales linearly with the number of preference vectors but can be reduced substantially through parallel training, as agents are fully independent. Importantly, this training cost is incurred offline only once. At inference time, the learned policies produce scheduling decisions with an average latency of ≈ 0.067 s per decision for the DS^{real}, which is negligible compared to typical workflow execution times. Therefore, the proposed approach maintains practical deployability while enabling broad Pareto-front coverage.

V. DISCUSSION

A. ANALYSIS OF RESEARCH QUESTIONS

This study addressed three key research questions regarding the design and effectiveness of the 2SD-GAT framework. The findings are summarized as follows:

RQ1: Two-stage DRL efficiency. Decomposing the scheduling problem into separate task-selection and VM-allocation stages reduced the action-space complexity from $O(N \times M)$ to $O(N) + O(M)$. This structural advantage enabled faster convergence and improved performance compared with heuristic and meta-heuristic baselines, the latter of which often produced weaker results while incurring substantially higher computation times.

RQ2: Multi-objective trade-offs. The preference-based reward formulation allowed the agent to effectively balance makespan, energy consumption, and QoS latency. Training with diverse preference vectors produced a broad Pareto front that outperformed baseline methods in both Hypervolume and IGD metrics. This confirms that the 2SD-GAT reward design successfully guides the agent toward diverse, high-quality solutions in a dynamic multi-objective setting. Sensitivity analysis further showed that the framework responds in a stable and interpretable manner to changes in preference weights.

RQ3: GNN-based dependency representation. The GNN encoder proved crucial for capturing the topological structure of workflow DAGs. Unlike fixed-input models, the GNN enabled generalization across workflows with varying shapes and complexities. The strong performance on unseen real-world traces demonstrates that the encoder successfully captured critical structural information. The ablation study further validated this design choice: replacing the GAT with either GIN or MLP led to weaker multi-objective performance, confirming the suitability of attention-based graph encoding within the two-stage DRL architecture.

B. COMPARATIVE ANALYSIS WITH EXISTING STUDIES

To contextualize the contributions of 2SD-GAT, a comparative analysis against several recent studies on cloud workflow scheduling that employ machine learning and reinforcement learning is provided in Table 17. The table summarizes key aspects, including optimization objectives, decision models, support for dynamic environments, structural modeling (e.g., DAG awareness), and Pareto-awareness.

As such, 2SD-GAT addresses a significant gap in the literature through the integration of a two-stage DRL and GNN-based process, in which task selection and resource allocation are separated and realized through a combination of GNN and MLP. Furthermore, preference-based reward tuning enables the method to be evaluated and applied across diverse settings, as reflected in the resulting Pareto front. Therefore, this constitutes a very promising avenue for future research.

C. STUDY LIMITATIONS AND FUTURE RESEARCH DIRECTIONS

However, despite its strengths, several limitations were observed during this research. A primary concern relates to the computational overhead of two-stage agents, as the two-stage PPO structure increases training time, particularly for large workflow DAGs with hundreds of tasks. As workflow size scales to thousands of tasks and large VM pools, this overhead can translate into increased inference latency, and in extreme-scale scenarios a simple heuristic may offer a more favorable cost–benefit trade-off. While this overhead is substantially lower than that of flat action-space approaches (e.g., [43]) and represents a common limitation among DRL-based schedulers, it nevertheless incurs a noticeable performance cost.

Another limitation arises from the use of fixed preference-based reward weights (λ_1, λ_2, λ_3) within each training instance. While this strategy is effective for generating a diverse set of Pareto-optimal solutions, it requires training multiple agents with different preference settings in order to cover the full trade-off space. A more flexible alternative would be to condition the policy on a preference vector provided as an input, enabling a single agent to adjust its behavior based on the desired objective weighting at inference time. Such an approach would allow a single agent to generalize across preference configurations, significantly reducing retraining requirements. Developing preference-conditioned policies is therefore a key direction for future work.

Furthermore, although both synthetic and real-world trace datasets were employed, the simulation environment necessarily abstracts several system-level factors, such as network latency, VM boot time, and interference from co-located jobs. These simplifications are common in workflow scheduling research, but incorporating them would bring the evaluation closer to the operational conditions of production cloud platforms. Future work will explore incorporating richer system-level dynamics, including network latency, VM boot delays, and interference from co-located workloads, to better approximate production cloud environments. The current QoS formulation is also simplified, as latency is modeled through a binary priority mechanism at the task start-time level. More expressive constraints, such as jitter bounds, burst deadlines, or percentile-based latency guarantees, could provide a more faithful representation of practical service-level expectations.

Another important direction for future work is modeling richer resource dynamics. While the study models VM unavailability through single-VM failure events, real cloud systems exhibit far richer forms of uncertainty, including fluctuating resource capacities, partial performance degradation, and correlated failures across hosts or racks. Capturing these behaviors would more accurately reflect the variability present in real-world environments. In addition, the framework currently assumes a fixed set of provisioned VMs. Extending it to include dynamic resource provisioning, such as elastic scaling or cost-aware VM acquisition, would allow the scheduler to reason jointly about task placement and resource expansion.

Another direction lies in extending the 2SD-GAT framework to diverse cloud-related environments, such as edge-cloud hybrids or edge computing, which would enhance

TABLE 17. Feature comparison of RL-based scheduling algorithms.

Algorithm/ Method	Workflow Scheduling (DAG)	Dynamic Scenarios	Makespan Objective	Energy Consumption Objective	QoS/SLA Objective	Real Data Inspired Datasets	Pareto-Based Comparisons	Compare with MO Algos	GNNs Utilized	PPO for Training	Two-stage Scheduling
DQTS [39]	✓	✓	✓	✗	✗	✓	✓	✗	✗	✗	✗
DRLB-TSA [54]	✗	✓	✓	✓	✓	✓	✗	✓	✗	✗	✗
DeepRM [46]	✗	✓	✗	✗	✗	✗	✗	✗	✗	✗	✗
DeepRM+ [47]	✗	✓	✗	✗	✗	✓	✗	✗	✗	✗	✗
ADRL [48]	✗	✓	✗	✗	✓	✓	✗	✗	✗	✗	✓
DRL-Cloud [40]	✓	✓	✓	✓	✓	✓	✗	✗	✗	✗	✓
LowCarbon [41]	✓	✓	✗	✓	✓	✓	✗	✗	✗	✗	✗
ODTS [55]	✗	✓	✗	✓	✓	✓	✗	✗	✗	✗	✓
DeepEnergyJSV2.0 [42]	✓	✓	✗	✓	✗	✓	✗	✗	✗	✓	✗
Liu et al. [44]	✗	✓	✗	✓	✓	✓	✗	✗	✗	✗	✓
MARL [4]	✓	✓	✓	✓	✗	✓	✗	✗	✗	✗	✓
IDRQN [50]	✓	✓	✗	✓	✓	✓	✗	✗	✗	✗	✗
DRL + FL [49]	✗	✓	✗	✓	✓	✓	✗	✗	✗	✗	✗
HITS [45]	✗	✓	✓	✗	✓	✓	✗	✗	✗	✗	✓
GNN-Flat [43]	✓	✓	✓	✓	✗	✗	✗	✗	✓	✓	✓
2SD-GAT	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

the applicability of the model to real-world multi-tenant scheduling scenarios.

Finally, while this study demonstrates strong performance relative to selected baselines, systematically benchmarking against a broader set of contemporary DRL scheduling methods under unified experimental settings remains an important direction for future research. Expanding comparisons to include diverse model architectures, training protocols, and workload characteristics would enable clearer cross-method generalization analysis and provide deeper insights into the relative strengths and limitations of different DRL-based scheduling paradigms. Such efforts would also support more standardized evaluation practices and facilitate fairer comparisons across future studies.

VI. CONCLUSION

Cloud workflow scheduling is a critical yet challenging NP-hard problem, requiring the simultaneous optimization of conflicting objectives such as makespan, energy consumption, and QoS latency under dynamic runtime conditions. Existing methods often fail to adequately model complex task dependencies or balance these multi-objective trade-offs effectively. To address this, this study proposed 2SD-GAT, a novel dynamic scheduling framework that integrates Two-Stage Deep Reinforcement Learning with Graph Neural Networks. By decomposing the scheduling problem into task selection and VM assignment stages, and utilizing a Graph Attention Network (GAT) to capture workflow structures, the proposed approach enables intelligent, adaptive decision-making.

Key findings from extensive evaluations on synthetic and real-world datasets demonstrate that 2SD-GAT significantly outperforms traditional heuristics and state-of-the-art DRL baselines. Specifically, it achieved superior convergence and diversity in Pareto-optimal solutions, with a recorded 26.8% gain in hypervolume and a 4.5x reduction in IGD in real-world traces under dynamic conditions, while maintaining low decision latency.

Despite limitations such as the increased training cost of the two-stage PPO architecture and the reliance on fixed preference weightings, the framework demonstrates strong potential for extensibility. Future research will explore preference-conditioned policies, richer models of system-level uncertainty, and dynamic resource provisioning, along with broader deployment scenarios such as edge-cloud environments. These directions aim to further strengthen the practical applicability of 2SD-GAT and enhance its capacity to operate in diverse, real-world cloud settings.

REFERENCES

[1] G. Andreadis, L. Versluis, F. Mastenbroek, and A. Iosup, "A reference architecture for datacenter scheduling: Extended technical report," 2018, *arXiv:1808.04224*.

[2] Y.-H. Hung, "Investigating how the cloud computing transforms the development of industries," *IEEE Access*, vol. 7, pp. 181505–181517, 2019.

[3] International Energy Agency. (2024). *Electricity 2024—Analysis and Forecast To 2026*. [Online]. Available: https://www.iea.org/reports/electricity-2024

[4] A. Jayanetti, S. Halgamuge, and R. Buyya, "Multi-agent deep reinforcement learning framework for renewable energy-aware workflow scheduling on distributed cloud data centers," *IEEE Trans. Parallel Distrib. Syst.*, vol. 35, no. 4, pp. 604–615, Apr. 2024.

[5] S. Cheikh and J. J. Walker, "Solving task scheduling problem in the cloud using a hybrid particle swarm optimization approach," *Int. J. Appl. Metaheuristic Comput.*, vol. 13, no. 1, pp. 1–25, Nov. 2021.

[6] L. Huang, J. Jia, B. Yu, B.-G. Chun, P. Maniatis, and M. Naik, "Predicting execution time of computer programs using sparse polynomial regression," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 23, 2010, pp. 883–891.

[7] S. Di, D. Kondo, and F. Cappello, "Characterizing and modeling cloud applications/jobs on a Google data center," *J. Supercomput.*, vol. 69, no. 1, pp. 139–160, Jul. 2014.

[8] E. H. Houssein, A. G. Gad, Y. M. Wazery, and P. N. Suganthan, "Task scheduling in cloud computing based on meta-heuristics: Review, taxonomy, open challenges, and future trends," *Swarm Evol. Comput.*, vol. 62, Apr. 2021, Art. no. 100841.

[9] Al. Green, H. Tai, J. Noffsinger, P. Sachdeva, A. Bhan, and R. Sharma, *How Data Centers and the Energy Sector Can Sate AI's Hunger for Power*, McKinsey & Company, New York, NY, USA, 2024.

[10] *Paris Agreement*, United Nations Framework Convention on Climate Change (UNFCCC), Bonn, Germany, 2015.

[11] J. Malmodin, N. Lövehagen, P. Bergmark, and D. Lundén, "ICT sector electricity consumption and greenhouse gas emissions—2020 outcome," *Telecommun. Policy*, vol. 48, no. 3, 2023, Art. no. 102701.

[12] D. Meedeniya, *Deep Learning: A Beginners' Guide*. CRC Press LLC, 2023.

[13] K. Lei, P. Guo, W. Zhao, Y. Wang, L. Qian, X. Meng, and L. Tang, "A multi-action deep reinforcement learning framework for flexible job-shop scheduling problem," *Expert Syst. Appl.*, vol. 205, Nov. 2022, Art. no. 117796.

[14] K. Lei, P. Guo, Y. Wang, J. Zhang, X. Meng, and L. Qian, "Large-scale dynamic scheduling for flexible job-shop with random arrivals of new jobs by hierarchical reinforcement learning," *IEEE Trans. Ind. Informat.*, vol. 20, no. 1, pp. 1007–1018, Jan. 2024.

[15] Y.-H. Chang, C.-H. Liu, and S. D. You, "Scheduling for the flexible job-shop problem with a dynamic number of machines using deep reinforcement learning," *Information*, vol. 15, no. 2, p. 82, Feb. 2024.

[16] E. Zitzler, K. Deb, and L. Thiele, "Comparison of multiobjective evolutionary algorithms: Empirical results," *Evol. Comput.*, vol. 8, no. 2, pp. 173–195, Jun. 2000.

[17] C. Audet, J. Bigeon, D. Cartier, S. Le Digabel, and L. Salomon, "Performance indicators in multiobjective optimization," *Eur. J. Oper. Res.*, vol. 292, no. 2, pp. 397–422, Jul. 2021.

[18] H. Topcuoglu, S. Hariri, and M.-Y. Wu, "Performance-effective and low-complexity task scheduling for heterogeneous computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 13, no. 3, pp. 260–274, Mar. 2002.

[19] S. Abrishami, M. Naghibzadeh, and D. H. J. Epema, "Deadline-constrained workflow scheduling algorithms for infrastructure as a service clouds," *Future Gener. Comput. Syst.*, vol. 29, no. 1, pp. 158–169, Jan. 2013.

[20] J. J. Durillo, H. M. Fard, and R. Prodan, "MOHEFT: A multi-objective list-based method for workflow scheduling," in *Proc. 4th IEEE Int. Conf. Cloud Comput. Technol. Sci.*, Dec. 2012, pp. 185–192.

[21] X. Zhou, G. Zhang, J. Sun, J. Zhou, T. Wei, and S. Hu, "Minimizing cost and makespan for workflow scheduling in cloud using fuzzy dominance sort based HEFT," *Future Gener. Comput. Syst.*, vol. 93, pp. 278–289, Apr. 2019.

[22] F. Juarez, J. Ejarque, and R. M. Badia, "Dynamic energy-aware scheduling for parallel task-based application in cloud computing," *Future Gener. Comput. Syst.*, vol. 78, pp. 257–271, Jan. 2018.

[23] N. Mansouri and R. Ghafari, "Cost-efficient task scheduling algorithm for reducing energy consumption and makespan of cloud computing," *Comput. Knowl. Eng.*, vol. 5, no. 1, pp. 1–12, 2022.

[24] L. Zhang, L. Wang, Z. Wen, M. Xiao, and J. Man, "Minimizing energy consumption scheduling algorithm of workflows with cost budget constraint on heterogeneous cloud computing systems," *IEEE Access*, vol. 8, pp. 205099–205110, 2020.

[25] R. Grandl, G. Ananthanarayanan, S. Kandula, S. Rao, and A. Akella, "Multi-resource packing for cluster schedulers," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 44, no. 4, pp. 455–466, Feb. 2015.

[26] H. Li, J. Li, Y. Wang, S. Nazarian, X. Lin, and Y. Wang, "Fast and energy-aware resource provisioning and task scheduling for cloud systems," in *Proc. 18th Int. Symp. Quality Electron. design (ISQED)*, Jul. 2017, pp. 174–179.

[27] S. Pandey, L. Wu, S. M. Guru, and R. Buyya, "A particle swarm optimization-based heuristic for scheduling workflow applications in cloud computing environments," in *Proc. 24th IEEE Int. Conf. Adv. Inf. Netw. Appl.*, Perth, WA, Australia, Apr. 2010, pp. 400–407.

[28] M. A. Rodriguez and R. Buyya, "Deadline based resource provisioningand scheduling algorithm for scientific workflows on clouds," *IEEE Trans. Cloud Comput.*, vol. 2, no. 2, pp. 222–235, Apr. 2014.

[29] P. Wang, Y. Lei, P. R. Agbedanu, and Z. Zhang, "Makespan-driven workflow scheduling in clouds using immune-based PSO algorithm," *IEEE Access*, vol. 8, pp. 29281–29290, 2020.

[30] Y. Wang and X. Zuo, "An effective cloud workflow scheduling approach combining PSO and idle time slot-aware rules," *IEEE/CAA J. Autom. Sinica*, vol. 8, no. 5, pp. 1079–1094, May 2021.

[31] B. Xiang, B. Zhang, and L. Zhang, "Greedy-ant: Ant colony system-inspired workflow scheduling for heterogeneous computing," *IEEE Access*, vol. 5, pp. 11404–11412, 2017.

[32] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002.

[33] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints," *IEEE Trans. Evol. Comput.*, vol. 18, no. 4, pp. 577–601, Aug. 2014.

[34] Y. Zhou and X. Jiao, "Knowledge-driven multi-objective evolutionary scheduling algorithm for cloud workflows," *IEEE Access*, vol. 10, pp. 2952–2962, 2022.

[35] L. Xing, J. Li, Z. Cai, and F. Hou, "Evolutionary optimization of energy consumption and makespan of workflow execution in clouds," *Mathematics*, vol. 11, no. 9, p. 2126, Apr. 2023.

[36] M. Shojafar, M. Kardgar, A. A. R. Hosseinabadi, S. Shamshirband, and A. Abraham, "TETS: A genetic-based scheduler in cloud computing to decrease energy and makespan," in *Proc. 15th Int. Conf. Hybrid Intell. Syst.*, Seoul, South Korea, 2015, pp. 103–115.

[37] X. Ye, J. Li, S. Liu, J. Liang, and Y. Jin, "A hybrid instance-intensive workflow scheduling method in private cloud environment," *Natural Comput.*, vol. 18, no. 4, pp. 735–746, Dec. 2019.

[38] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Comput.*, vol. 11, no. 6, pp. 712–731, Dec. 2007.

[39] Z. Tong, H. Chen, X. Deng, K. Li, and K. Li, "A scheduling scheme in the cloud computing environment using deep Q-learning," *Inf. Sci.*, vol. 512, pp. 1170–1191, Feb. 2020.

[40] M. Cheng, J. Li, and S. Nazarian, "DRL-cloud: Deep reinforcement learning-based resource provisioning and task scheduling for cloud service providers," in *Proc. 23rd Asia South Pacific Design Autom. Conf. (ASP-DAC)*, Jul. 2018, pp. 129–134.

[41] W. Liu, Y. Yan, Y. Sun, H. Mao, M. Cheng, P. Wang, and Z. Ding, "Online job scheduling scheme for low-carbon data center operation: An information and energy Nexus perspective," *Appl. Energy*, vol. 338, May 2023, Art. no. 120918.

[42] Y. Yang, C. He, B. Yin, Z. Wei, and B. Hong, "Cloud task scheduling based on proximal policy optimization algorithm for lowering energy consumption of data center," *KSII Trans. Internet Inf. Syst. (TIIS)*, vol. 16, no. 6, pp. 1877–1891, 2022.

[43] S. Chandrasiri and D. Meedeniya, "Energy-efficient dynamic workflow scheduling in cloud environments using deep learning," *Sensors*, vol. 25, no. 5, p. 1428, Feb. 2025.

[44] N. Liu, Z. Li, J. Xu, Z. Xu, S. Lin, Q. Qiu, J. Tang, and Y. Wang, "A hierarchical framework of cloud resource allocation and power management using deep reinforcement learning," in *Proc. IEEE 37th Int. Conf. Distrib. Comput. Syst. (ICDCS)*, Jun. 2017, pp. 372–382.

[45] D. Cui, Z. Peng, K. Li, Q. Li, J. He, and X. Deng, "An novel cloud task scheduling framework using hierarchical deep reinforcement learning for cloud computing," *PLoS ONE*, vol. 20, no. 8, Aug. 2025, Art. no. e0329669.

[46] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, "Resource management with deep reinforcement learning," in *Proc. 15th ACM workshop hot topics Netw.*, 2016, pp. 50–56.

[47] W. Guo, W. Tian, Y. Ye, L. Xu, and K. Wu, "Cloud resource scheduling with deep reinforcement learning and imitation learning," *IEEE Internet Things J.*, vol. 8, no. 5, pp. 3576–3586, Mar. 2021.

[48] S. Kardani-Moghaddam, R. Buyya, and K. Ramamohanarao, "ADRL: A hybrid anomaly-aware deep reinforcement learning-based resource scaling in clouds," *IEEE Trans. Parallel Distrib. Syst.*, vol. 32, no. 3, pp. 514–526, Mar. 2021.

[49] N. Shan, X. Cui, and Z. Gao, "'DRL + FL': An intelligent resource allocation model based on deep reinforcement learning for mobile edge computing," *Comput. Commun.*, vol. 160, pp. 14–24, Jul. 2020.
[50] H. Lu, C. Gu, F. Luo, W. Ding, and X. Liu, "Optimization of lightweight task offloading strategy for mobile edge computing based on deep reinforcement learning," *Future Gener. Comput. Syst.*, vol. 102, pp. 847–861, Jan. 2020.
[51] C. F. Hayes, R. Rădulescu, E. Bargiacchi, J. Källström, M. Macfarlane, M. Reymond, T. Verstraeten, L. M. Zintgraf, R. Dazeley, F. Heintz, E. Howley, A. A. Irissappane, P. Mannion, A. Nowé, G. Ramos, M. Restelli, P. Vamplew, and D. M. Roijers, "A practical guide to multi-objective reinforcement learning and planning," *Auto. Agents Multi-Agent Syst.*, vol. 36, no. 1, p. 26, Apr. 2022.
[52] S. Mohammadi, L. PourKarimi, and H. Pedram, "Integer linear programming-based multi-objective scheduling for scientific workflows in multi-cloud environments," *J. Supercomput.*, vol. 75, no. 10, pp. 6683–6709, Oct. 2019.
[53] M. Hussain, L.-F. Wei, A. Lakhan, S. Wali, S. Ali, and A. Hussain, "Energy and performance-efficient task scheduling in heterogeneous virtualized cloud computing," *Sustain. Comput., Informat. Syst.*, vol. 30, Jun. 2021, Art. no. 100517.
[54] M. Sudheer, K. Reddy, M. Kumar, O. Khalaf, C. Romero, and G. Sahib, "DRLBTSA: Deep reinforcement learning based task-scheduling algorithm in cloud computing," *Multimedia Tools Appl.*, vol. 83, no. 3, pp. 8359–8387, Jan. 2024.
[55] J. Lou, Z. Tang, and W. Jia, "Energy-efficient joint task assignment and migration in data centers: A deep reinforcement learning approach," *IEEE Trans. Netw. Service Manage.*, vol. 20, no. 2, pp. 961–973, Jun. 2023.
[56] K. Lei, P. Guo, Y. Wang, J. Xiong, and W. Zhao, "An end-to-end hierarchical reinforcement learning framework for large-scale dynamic flexible job-shop scheduling problem," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Padua, Italy, Jul. 2022, pp. 1–8.
[57] R. S. Sutton and A. Barto, "Reinforcement learning: An introduction," *Bradford Book*, vol. 9, no. 5, p. 1054, 1998.
[58] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," 2017, *arXiv:1707.06347*.
[59] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," 2017, *arXiv:1710.10903*.
[60] P. Erdõs and A. Rényi, "On random graphs. I," *Publ. Math. Debrecen*, vol. 6, nos. 3–4, pp. 290–297, 2022.
[61] D. Cordeiro, G. Mounié, S. Perarnau, D. Trystram, J.-M. Vincent, and F. Wagner, "Random graph generation for scheduling simulations," in *Proc. 3rd Int. ICST Conf. Simul. Tools Techn.*, 2010, pp. 1–10, Art. no. 60.
[62] Standard Performance Evaluation Corporation. (2008). *SPECpower_SSJ 2008 Results*. [Online]. Available: https://www.spec.org/power_ssj2008/results/
[63] J. Wilkes, "Google cluster-usage traces V3," Google LLC, CA, USA, Tech. Rep. ClusterData 2019, 2020.
[64] W. M. van Der Aalst, A. H. Ter Hofstede, B. Kiepuszewski, and A. P. Barros, "Workflow patterns," *Distrib. Parallel Databases*, vol. 14, no. 1, pp. 5–51, 2003.
[65] S. Fu, "Failure-aware resource management for high-availability computing clusters with distributed virtual machines," *J. Parallel Distrib. Comput.*, vol. 70, no. 4, pp. 384–393, Apr. 2010.
[66] S. Huang, R. F. J. Dossa, C. Ye, and J. Braga, "CleanRL: High-quality single-file implementations of deep reinforcement learning algorithms," *J. Mach. Learn. Res.*, vol. 23, no. 274, pp. 1–18, 2021.
[67] M. Kumar, S. C. Sharma, A. Goel, and S. P. Singh, "A comprehensive survey for scheduling techniques in cloud computing," *J. Netw. Comput. Appl.*, vol. 143, pp. 1–33, Oct. 2019.
[68] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" 2018, *arXiv:1810.00826*.



SUNERA CHANDRASIRI received the B.Sc. degree (Hons.) in engineering from the University of Moratuwa, Sri Lanka, where he is currently pursuing the M.Sc. (by Research) degree with the Department of Computer Science and Engineering. His research interests include machine learning, reinforcement learning, cloud scheduling, programming languages, and bioinformatics. He is a member of the Institution of Engineers, Sri Lanka (IESL).



DULANI MEEDENIYA (Senior Member, IEEE) received the Ph.D. degree in computer science from the University of St Andrews, U.K. She is currently a Professor of computer science and engineering with the University of Moratuwa, Sri Lanka. She is also the Director of the Bio-Health Informatics Group and the Eco-Sustainable Informatics Group at her department and engages in many collaborative research. She is the co-author of more than 150 publications in indexed journals, peer-reviewed conferences, and book chapters. Her main research interests include software engineering, artificial intelligence, deep learning, computer vision, and the Internet of Things across multidisciplinary domains. She is a fellow of HEA, U.K., MIET, a member of ACM, and a Chartered Engineer registered at EC, U.K. She has received several awards, including the President's Award for Scientific Research and grants. She serves as a reviewer, a program committee member, and an editor for many international venues. She has recognized in the Stanford and Elsevier Top 2% Scientists List 2025, that highlights the world's leading scientists who have made a substantial impact.